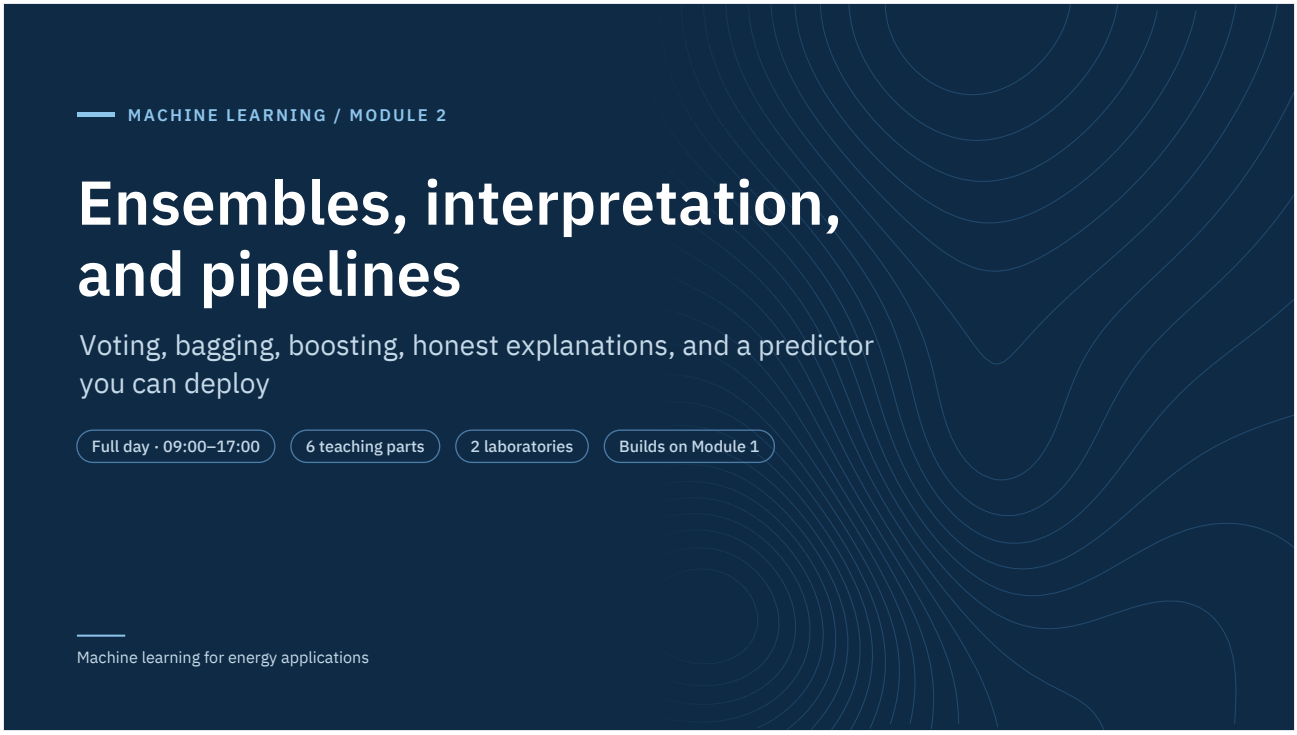
Ensembles, interpretation, and pipelines



Welcome learners and state the prerequisite: Module 1, including leakage, grouped validation, pipelines, and single decision trees. Today moves from one model to many models, then asks how to explain them and how to ship them safely.

Warm-up question for the room while people settle [2 min]: "Name one reason a single deep tree might change a lot if you retrained it on next month's data." Collect two or three answers (small data, noisy labels, greedy splits, many near-equal split candidates). Keep them on the whiteboard; we return to them in Part 2 when we show trees fitted to bootstrap samples.

Practical notes: all energy datasets on the slides are synthetic and labelled as such in the captions. The laboratory notebook uses the small Iris dataset so that code runs in seconds; the energy examples carry the domain lessons. Transition: the agenda for the day.

Today's plan



Walk through the day in one minute. The morning builds the two big ensemble families: bagging (random forests) and boosting (AdaBoost and gradient boosting). Lab 1 compares a single tree, a forest, and a boosted model on identical folds, so the afternoon can build on real results.

The afternoon covers the engineering side of boosting (histogram libraries, missing values, categories), stacking, interpretation (importance, partial dependence, Shapley values), and packaging (pipelines, leakage, saved artifacts, model cards). Lab 2 applies importance, pipelines, and tuning in the same notebook.

Timing guide: each lecture block ends with an exercise or check question; answers are in these notes. If a block overruns, shorten the code walk-throughs rather than the exercises. Hyperparameter search strategies, nested cross-validation, calibration, and drift detection are Module 5 topics; today only previews them. Transition: learning outcomes.

What you will be able to do

LEARNING OUTCOMES

What you will be able to do

By the end of the day you will be able to:

1. Explain when voting or averaging reduces error, using member accuracy and correlation.
2. Fit and tune random forests, and weigh out-of-bag scores against grouped validation.
3. Trace AdaBoost and gradient-boosting updates and control them with depth, learning rate, and early stopping.
4. Choose between a forest, boosting, and stacking under a validation and deployment budget.
5. Interpret an ensemble with permutation importance, PDP/ICE, and Shapley values, and state their limits.
6. Package preprocessing and model in one pipeline, avoid leakage, and save it with its context.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 3

Read the outcomes aloud and point out that each is observable: learners will compute a majority-vote accuracy, read an out-of-bag score against a grouped score, perform a boosting update by hand, compute Shapley values by hand, and produce a saved pipeline that reloads with identical predictions.

Emphasize the thread that runs through the whole day: every ensemble, explanation, and pipeline is only as trustworthy as the validation design behind it. The same grouped or chronological splits from Module 1 apply to ensembles unchanged.

Assessment evidence for the day: the Lab 1 comparison table, the Lab 2 importance table and reload check, and the exit check at 16:45. Ask one person to restate the difference between a model that is accurate and a model whose evaluation is trustworthy. Transition: a warm-up exercise with three classifiers.

A fragile model can become a useful component

PRACTICE

4 min Pairs

A fragile model can become a useful component

Three classifiers produce event probabilities 0.2, 0.7, and 0.6.

1. What is their mean probability?
2. What prediction results from a 0.5 threshold?
3. What if all three learned the same systematic error?

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 4

[4 min] Give pairs two minutes, then collect answers.

Answers. (1) Mean probability = (0.2 + 0.7 + 0.6) / 3 = 1.5 / 3 = 0.5. (2) The outcome depends on the threshold convention: with "predict event if p ≥ 0.5" the average predicts event; with a strict "p > 0.5" it does not. A majority vote of the three hard decisions (no, yes, yes) predicts event. (3) If all three share the same systematic error, for example all miss events in one formation, averaging cannot remove it: identical errors survive any average.

Common error: assuming the average of probabilities and the majority of votes always agree. They agree here only because the mean sits exactly at the threshold; the next part shows a case where they disagree.

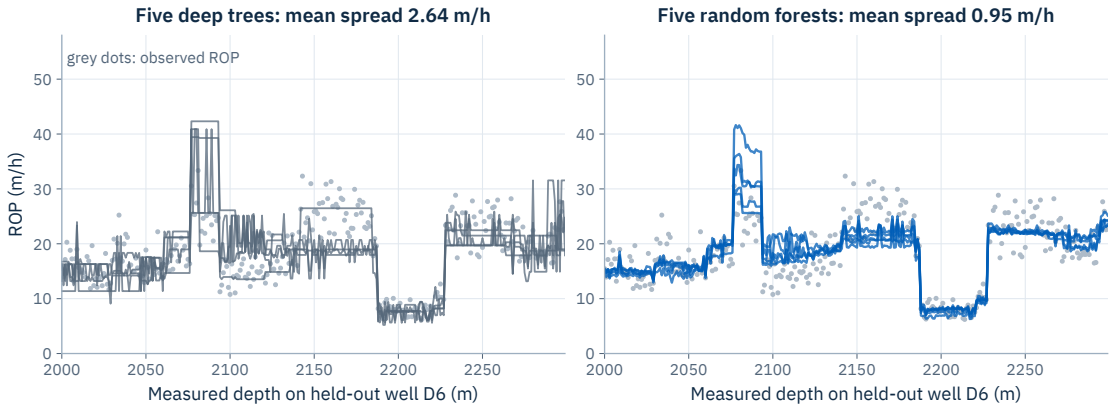
Ask: why can identical copies of one model not add information? Because the ensemble gains only from errors that differ. Transition: a drilling case where this matters.

Retrained trees disagree; retrained forests agree

MACHINE LEARNING / MODULE 2

Retrained trees disagree; retrained forests agree

Five resamples of wells D1–D5; every model predicts rate of penetration (ROP) on well D6.



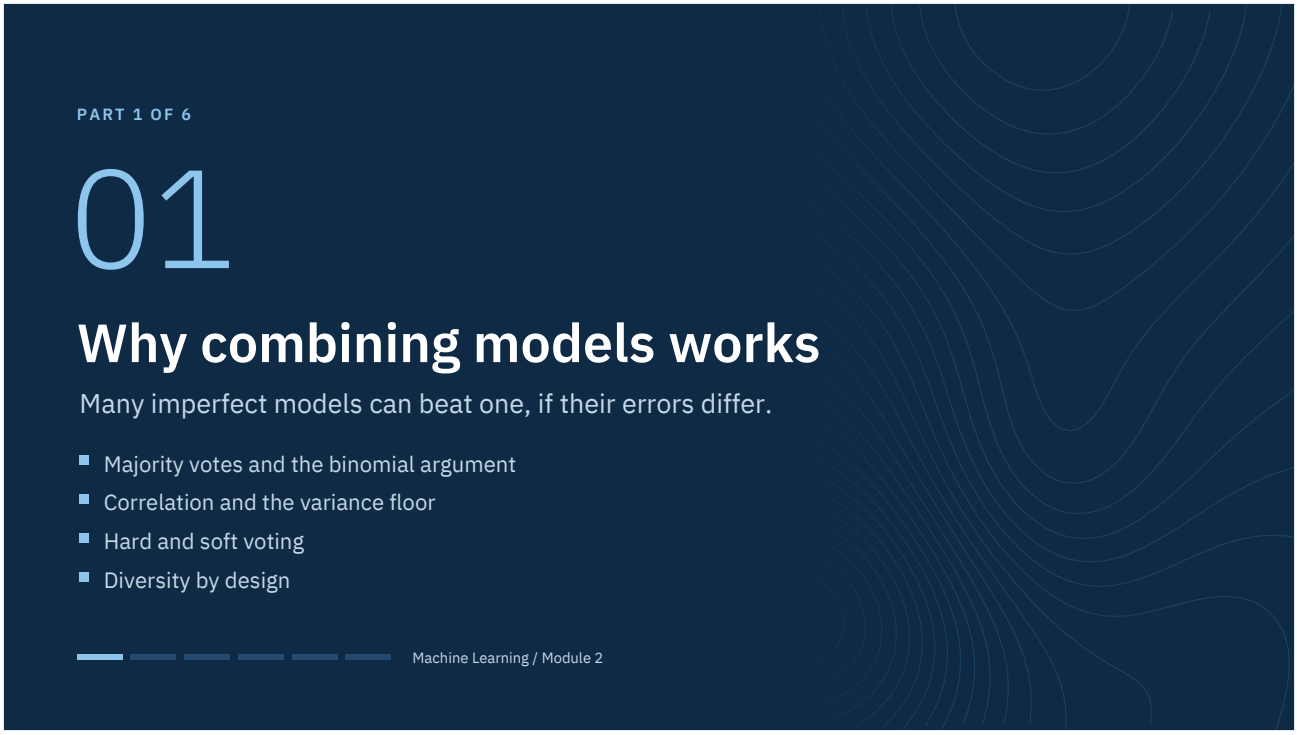
Synthetic drilling data (seed 11). Spread = standard deviation across the five models, averaged over depth: trees 2.64 m/h, forests 0.95 m/h.

Motivating case [3 min]. We resampled the training wells five times (bootstrap samples of rows) and fitted two model types on each resample: one deep regression tree and one random forest. Each model then predicted ROP along held-out well D6.

Left: the five trees tell different stories at the same depth, sometimes jumping by 10 m/h between neighbouring metres. Right: the five forests nearly overlap. The average spread across models is 2.64 m/h for trees and 0.95 m/h for forests. Neither panel proves accuracy; it shows stability. A planner looking at the tree predictions would get a different drilling-performance story each time the model is retrained.

Ask: which would you rather put in front of a drilling engineer? And: what would you still need before trusting either? (Error on several new wells, not one; errors by formation; a baseline.) The data are synthetic and generated from an illustrative formula, so no field performance is implied. Transition: Part 1 explains why averaging works.

Why combining models works



Part 1 [35 min]. The goal is intuition with exact numbers: first for votes (classification), then for averages (regression), then for the practical choice between hard and soft voting, and finally for the ways we create diverse members.

Key message to plant now: combining models is not magic. It helps only when members are individually useful and do not all make the same mistakes. Every later method (bagging, random forests, boosting, stacking) is a different way to create useful members with different errors.

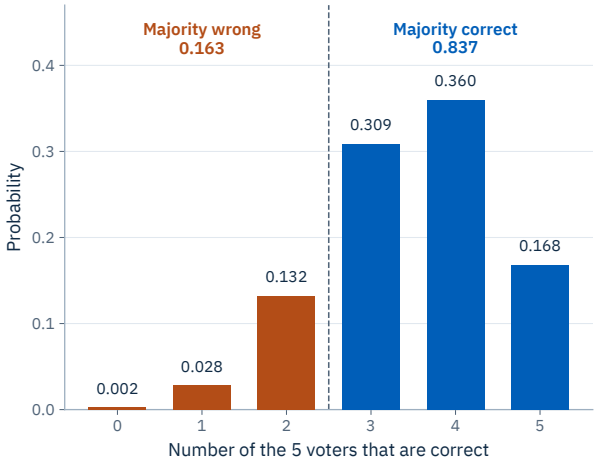
Ask the room before starting: if five colleagues each guess a formation top correctly 70% of the time, how often is the majority right? Take a few guesses and write them down; the next slide computes the answer exactly. Transition: the binomial argument.

Independent errors rarely line up

WHY COMBINING MODELS WORKS

Independent errors rarely line up

- Five voters, each right 70% of the time, with independent errors.
- The vote fails only when three or more are wrong: probability 0.163.
- Majority accuracy is 0.837, better than any single member.



Exact binomial probabilities for n = 5 voters, each correct with p = 0.7.

◆ Independence is the assumption doing the work.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 7

[4 min] Walk through the bars. The number of correct voters out of five follows a binomial distribution. The majority is right when at least three voters are right: $P = C(5,3)\cdot 0.7^3\cdot 0.3^2 + C(5,4)\cdot 0.7^4\cdot 0.3 + 0.7^5 = 0.3087 + 0.3602 + 0.1681 = 0.8369$.

Compare with the guesses collected at the section slide; most people underestimate the gain.

Now attack the assumption. Independent errors mean that one voter being wrong tells you nothing about the others. Real models trained on the same data share blind spots: the same mislabelled rows, the same missing formation, the same sensor drift. When errors are correlated, the bars on the right shrink and the gain disappears.

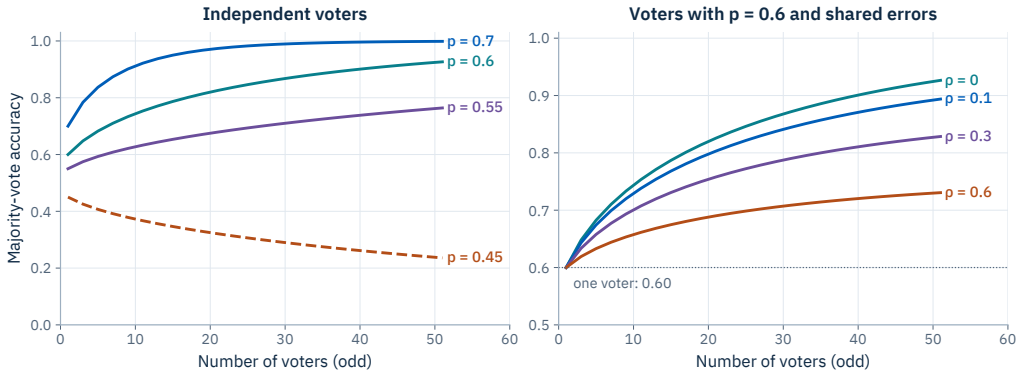
Ask: what would independence mean for three facies classifiers trained on the same eight wells? (Hardly achievable; we can only reduce the correlation.) Transition: how the gain grows with the number of voters and how correlation caps it.

Voting helps only accurate, independent members

WHY COMBINING MODELS WORKS

Voting helps only accurate, independent members

Left: members above 50% improve with numbers, members below 50% get worse. Right: shared errors cap the gain.



Exact binomial majority probabilities. Right: with probability ρ all voters copy one shared vote; otherwise they vote independently.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 8

[4 min] Left panel: majority accuracy against the number of voters for members that are right 55%, 60%, or 70% of the time, and a dashed curve for 45%. With independent errors, even weak members above chance approach perfect accuracy as the number grows; members below chance approach zero. Averaging amplifies whichever side of chance the members are on.

Right panel: members at 60%, with a shared-error model. With probability ρ all voters copy one shared vote; otherwise they vote independently. This gives a pairwise correlation of correctness equal to ρ, and the accuracy is ρ·p + (1 − ρ)·P(independent majority). With ρ = 0.3 and 51 voters the curve reaches 0.83 instead of 0.93, and it can never exceed ρ·0.6 + (1 − ρ) = 0.88.

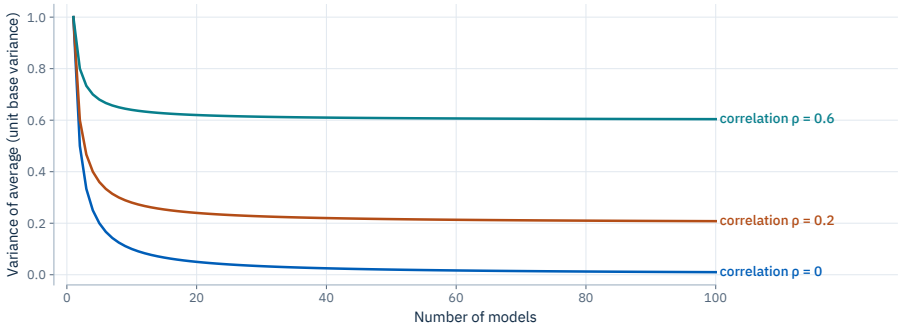
Common misconception: "more models always help". More models help only while members stay useful and their errors stay different. Transition: the same idea for averaged numbers.

Correlation leaves a variance floor

WHY COMBINING MODELS WORKS

Correlation leaves a variance floor

$$\mathrm{Var}(\bar{f}) = \sigma^2\left(\rho + \frac{1-\rho}{M}\right)$$



Analytical curves for unit model variance and several assumed error correlations; not a measured benchmark. Example: ρ = 0.2, M = 10 gives 0.28; the floor is 0.20.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 9

[4 min] The formula describes M models with equal variance σ² and a common pairwise correlation ρ between their errors. The average has variance ρσ² + (1 − ρ)σ²/M. The second term shrinks as M grows; the first does not. For variance 1, correlation 0.2, and ten models: 0.2 + 0.8/10 = 0.28. With a thousand models: 0.2 + 0.0008 = 0.2008, essentially the floor ρσ² = 0.2.

This idealized formula is not a statement about any fitted forest; it explains the design goal. Increasing M alone cannot remove shared bias or leakage. Lowering ρ, the correlation, is what moves the floor, which is exactly what random forests do with feature subsampling.

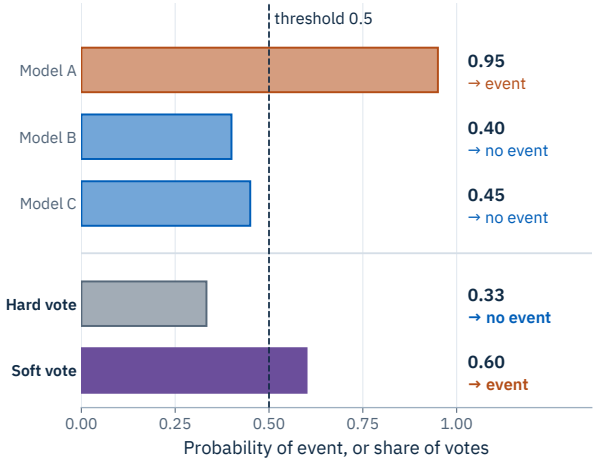
Trace the high-correlation line and ask what intervention might lower it. Good answers: different bootstrap samples, random feature subsets, different model families. Warn that diversity is only useful if the members still contain predictive information. Transition: two ways to combine classifiers.

Hard and soft voting can disagree

WHY COMBINING MODELS WORKS

Hard and soft voting can disagree

- Hard vote: each model casts one vote for its predicted class.
- Soft vote: average the predicted probabilities, then apply the threshold.
- Soft voting uses confidence, so members need sensible probabilities.



◆ Same three models, opposite decisions.

Exact arithmetic for three illustrative probabilities.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 10

[3 min] Three models give event probabilities 0.95, 0.40, and 0.45. Hard voting counts decisions: one model says event, two say no event, so the share of votes for event is 1/3 = 0.33 and the ensemble predicts no event. Soft voting averages probabilities: (0.95 + 0.40 + 0.45) / 3 = 0.60, so the ensemble predicts event.

Which is right depends on whether the confidence of model A is trustworthy. Soft voting rewards a very confident member; that is good when probabilities are well calibrated and harmful when one model is systematically over-confident. Calibration is covered in Module 5; for now, check reliability on validation data before choosing soft voting.

Energy example: three alarm models for an electrical submersible pump. One model is almost certain of an event, two are mildly against. Ask which rule you would want, and what evidence you would need to trust the confident model. Transition: the scikit-learn tool.

VotingClassifier mixes model families

WHY COMBINING MODELS WORKS

VotingClassifier mixes model families

```python
from sklearn.ensemble import VotingClassifier
members = [
    ("logit", make_pipeline(StandardScaler(), LogisticRegression())),
    ("knn", make_pipeline(StandardScaler(), KNeighborsClassifier(15))),
    ("forest", RandomForestClassifier(300, random_state=0)),
]
vote = VotingClassifier(members, voting="soft")

cv = GroupKFold(n_splits=4)                  # new wells in every fold
for name, model in members + [("soft vote", vote)]:
    s = cross_val_score(model, X, y, groups=wells, cv=cv)
    print(f"{name:10s} {s.mean():.3f}")
```

OUTPUT

```
logit      0.857
knn        0.813
forest     0.776
soft vote  0.841
```

◆ The vote trails its best member here: combining is a hypothesis to test.

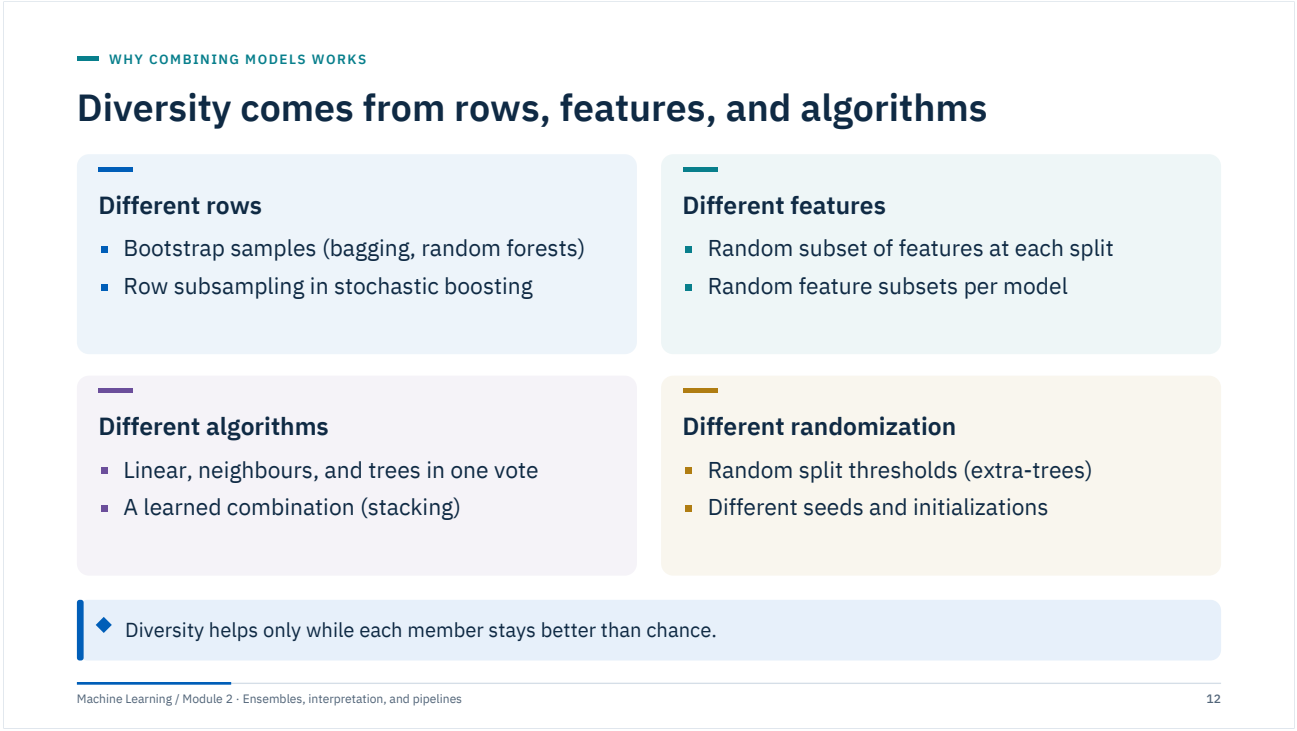
Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 11

[4 min] Context: X holds the four logs (GR, RHOB, NPHI, PE) of the synthetic eight-well facies dataset, y the facies, wells the well names; imports as in Module 1. Every candidate, including the ensemble, is scored with the same grouped folds, so each fold predicts wells it never saw.

Read the output honestly. The logistic regression scores 0.857 on new wells, the soft vote 0.841, and the forest only 0.776: the forest learns well-specific patterns that do not transfer. The vote beats two of its members but not the best one. (A hard vote scores 0.828 on the same folds.)

Lessons: voting is cheap to try and often stabilizes results, but it is not guaranteed to beat the best member; members that are much weaker drag the average down. Soft voting requires every member to implement predict_proba. Weights can be passed with the weights argument, but choosing weights is itself tuning and must happen inside validation. Transition: where diversity comes from.

Diversity comes from rows, features, and algorithms



[3 min] This slide is a map for the rest of the day. Every ensemble method we meet creates diversity in at least one of these four ways.

Different rows: bagging and random forests train each tree on a bootstrap sample; stochastic gradient boosting fits each tree on a random fraction of rows. Different features: random forests draw a new random subset of candidate features at every split; the random subspace method fixes one subset per model. Different algorithms: voting and stacking combine model families whose errors come from different assumptions (linear boundaries, local neighbourhoods, axis-aligned splits). Different randomization: extra-trees also randomize split thresholds; neural-network ensembles differ in initialization.

Link back to the curves: diversity lowers the correlation ρ, but every mechanism also weakens individual members a little. The best ensembles balance the two, which is the trade-off we will measure for max_features in Part 2. Transition: a short check before moving on.

How much can three voters gain?

PRACTICE

5 min Pairs

How much can three voters gain?

Assume independent errors unless stated otherwise.

1. Three classifiers are each correct 80% of the time. What is the majority-vote accuracy?
2. Repeat for three classifiers that are each correct 40% of the time.
3. What is the accuracy if the three are identical copies of one 80% model?

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 13

[5 min] Answers with arithmetic. For three voters the majority is right when at least two are right.

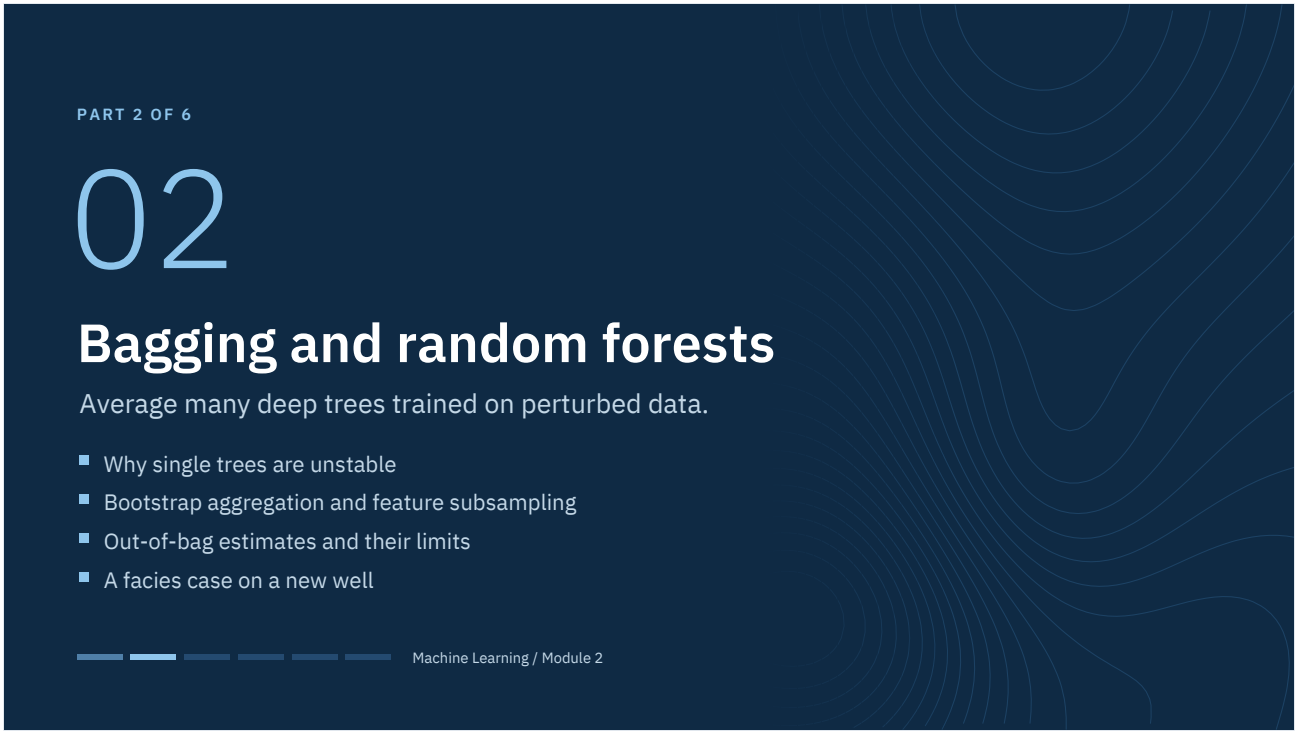
(1) p = 0.8: $P = 3 \cdot 0.8^2 \cdot 0.2 + 0.8^3 = 3 \cdot 0.64 \cdot 0.2 + 0.512 = 0.384 + 0.512 = 0.896$.

(2) p = 0.4: $P = 3 \cdot 0.4^2 \cdot 0.6 + 0.4^3 = 0.288 + 0.064 = 0.352$, worse than a single member (0.40). Voting amplifies whichever side of chance the members are on.

(3) Identical copies always agree, so the vote equals one model: 0.80. No gain at all.

Common errors: forgetting the combinatorial factor 3 (there are three ways to choose which voter is wrong); treating $0.8^3 = 0.512$ as the answer (that is "all three right", not "majority right"). Extension for fast pairs: with five voters at p = 0.8 the answer is 0.942. Ask: which of the three scenarios best describes three random forests trained with different seeds on the same data? (Closest to case 3: highly correlated.) Transition: Part 2, bagging.

Bagging and random forests



Part 2 [45 min]. We start from the instability learners named in the warm-up: a deep tree is a high-variance estimator. Bagging (bootstrap aggregating, Breiman 1996) reduces that variance by averaging trees fitted to resampled data; random forests (Breiman 2001) decorrelate the trees further by restricting the features searched at each split.

The second half of the part is about evidence. Forests come with a free out-of-bag estimate, and it is tempting to report it. We will show when it is trustworthy and when rows from the same well make it optimistic.

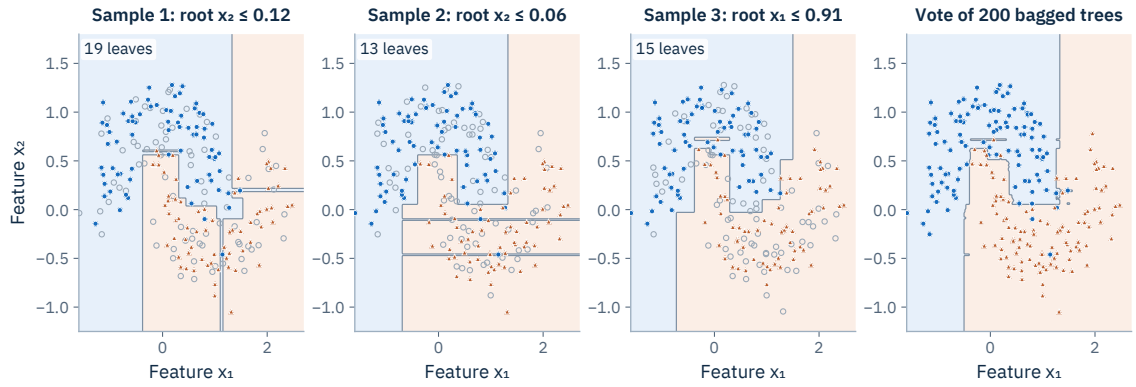
The energy case at the end classifies facies on a held-out well, the typical deployment question for log-based models. Transition: trees fitted to three bootstrap samples.

A deep tree changes with every bootstrap sample

BAGGING AND RANDOM FORESTS

A deep tree changes with every bootstrap sample

Same 220 rows, three bootstrap resamples: different root splits, leaf counts, and boundaries.



Synthetic two-moons data (seed 42); bootstrap seed 18; unpruned scikit-learn trees. Hollow markers: rows left out of that sample.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 15

[3 min] Each of the first three panels is an unpruned tree fitted to a bootstrap sample of the same 220 rows. Look at the titles: the root split is $x_2 \le 0.12$, then $x_2 \le 0.06$, then $x_1 \le 0.91$, and the trees have 19, 13, and 15 leaves. Small changes in the sample change the first split, and every later split depends on the first one. That is why trees are called unstable, high-variance learners.

The fourth panel is the vote of 200 such trees (a bagged ensemble). Its boundary is not smooth, but it no longer depends on which rows happened to be drawn; the isolated islands that each single tree creates around a few points mostly disappear.

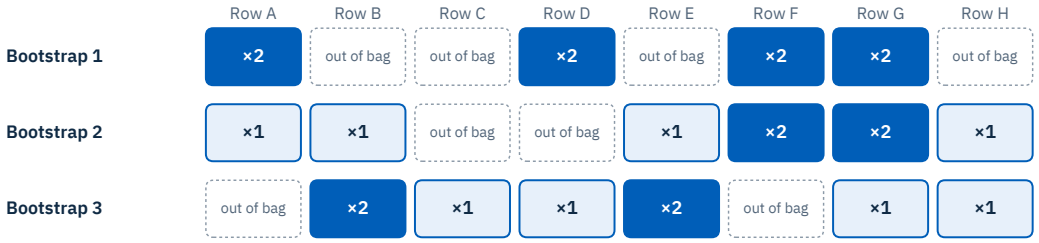
Link to the warm-up answers: greedy splitting, many near-equal candidates, and noisy points all contribute. Ask: is instability a bug or a feature for an ensemble? It is a feature: unstable members give diverse errors, which averaging can cancel. Stable members such as linear models gain little from bagging. Transition: what a bootstrap sample looks like.

Bootstrap samples contain duplicates

BAGGING AND RANDOM FORESTS

Bootstrap samples contain duplicates

A row drawn twice counts twice in that tree's fit; rows never drawn are its out-of-bag rows.



Each bootstrap draws 8 rows with replacement: some rows repeat, others are left out (out of bag).

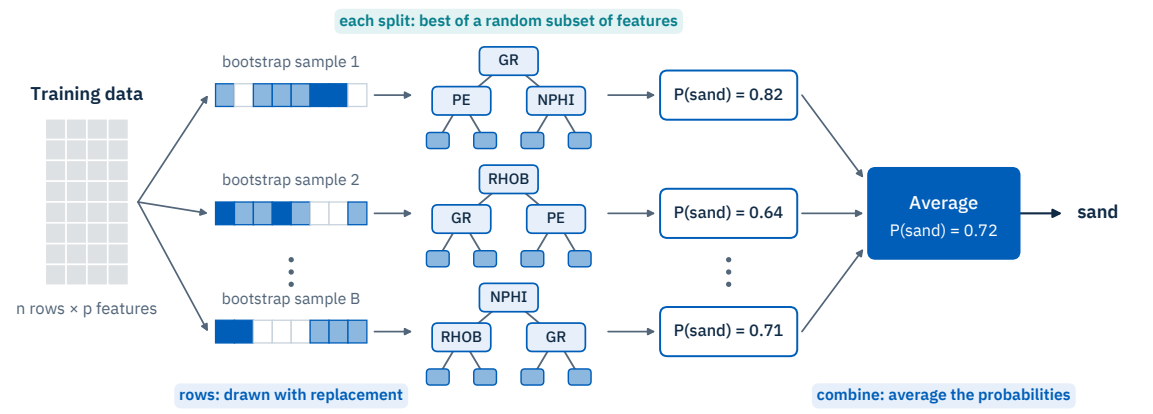
One small synthetic bootstrap example (seed 42), n = 8 rows.

[3 min] Ask learners to find a row repeated in one sample and a row left out. In bootstrap 1, rows A, D, F, and G appear twice and rows B, C, E, and H are out of bag. In bootstrap 2, rows F and G appear twice and C and D are out of bag.

A bootstrap sample has the original number of rows but not the same number of unique rows. The chance that a given row is never drawn in n draws is $(1 - 1/n)^n$. For n = 8 that is $(7/8)^8 = 0.344$; as n grows it approaches $e^{-1} = 0.368$, so roughly 36.8% of rows are out of bag for each tree. Bootstrap 1 left out 4 of 8 rows, more than expected: small samples vary. Across the three samples, 8 of 24 cells are out of bag (33%).

Keep this picture in mind: the out-of-bag rows are a free validation set for each tree, which we exploit later in this part. Transition: the full random-forest recipe.

A random forest decorrelates trees before voting



[3 min] Walk left to right. (1) Draw B bootstrap samples of the training rows. (2) Grow one deep tree per sample; at every split, the tree may only choose among a random subset of the features, size max_features. (3) Combine: regression forests average the trees' predictions; scikit-learn classifiers average the trees' class probabilities and predict the class with the highest mean probability.

The feature restriction is what distinguishes a random forest from plain bagging. Without it, every tree would usually split on the strongest log (say PE) near the root, and the trees would be highly correlated. Forcing some trees to split on GR or NPHI first produces different, still useful trees, which lowers ρ in the variance formula.

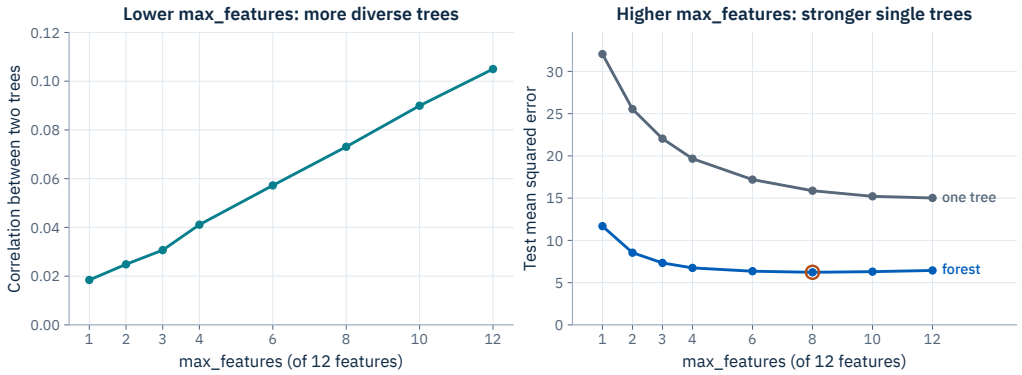
Note on terms: "votes" in many textbooks; scikit-learn's classifier averages probabilities, which is a soft vote. Ask: what does max_features = all features give? Plain bagging of trees. Transition: measuring the trade-off controlled by max_features.

Feature subsampling trades strength for diversity

BAGGING AND RANDOM FORESTS

Feature subsampling trades strength for diversity

Fewer features per split: less correlated trees but weaker ones. The forest does best in between.



Friedman #1 synthetic regression (12 features, 5 informative): 16 independent training sets of 300 rows, forests of 60 trees. Circle: lowest forest error.

[4 min] Left: the correlation between two trees of the same forest, estimated as in Hastie, Tibshirani and Friedman (chapter 15): the share of a single tree's variance that comes from the training set rather than from the forest's randomization. It rises from 0.018 with one feature per split to 0.105 with all twelve. Right: single trees get much better as they may choose among more features (test MSE 32.0 down to 15.0), while the forest's error falls from 11.7 to a minimum of 6.2 at eight features and then rises slightly to 6.4 with all twelve (plain bagging).

The lesson is the trade-off, not the specific optimum: the best max_features depends on how many features are informative. With many noise features, very small values starve trees of signal; with a few strong, redundant features, smaller values help. scikit-learn's defaults are max_features = "sqrt" for classifiers and 1.0 (all features) for regressors; both are starting points to tune, not answers.

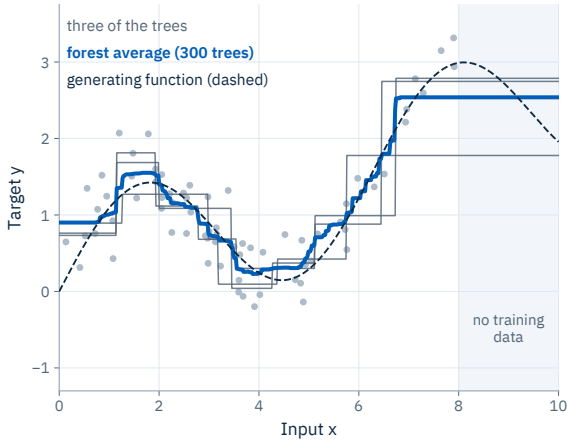
Ask: why is the forest always better than one tree here? Averaging removes the variance part of the tree's error. Transition: what averaging looks like in regression.

Averaging turns jagged trees into a smoother fit

BAGGING AND RANDOM FORESTS

Averaging turns jagged trees into a smoother fit

- Each tree is a step function fitted to its own bootstrap sample.
- The forest averages 300 step functions: steps blur and noise cancels.
- Beyond the data (shaded) the average stays flat: trees cannot extrapolate.



◆ Forests average predictions; classifiers average class probabilities.

Synthetic data y = sin(x) + 0.25x + noise, n = 70 on 0 ≤ x ≤ 8 (seed 8); 300 trees, min_samples_leaf = 5.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 19

[3 min] Grey lines are three of the 300 trees; each is a piecewise-constant function whose steps sit at different places because each tree saw a different bootstrap sample. The blue forest average is a much finer staircase that follows the dashed generating function more closely than any single tree.

Point at the shaded region (x > 8), where there are no training rows: the true function falls from 3.0 to 2.0, but the forest stays flat at about 2.5, because every leaf predicts an average of training targets and nothing beyond the last training points can be learned. Trees, forests, and boosted trees do not extrapolate trends. In energy applications this matters for production forecasting beyond the historical range, higher rates than ever seen, or a new pressure regime: the model will predict the edge values.

Ask: what would you add to a forest if a trend must be extrapolated? (A physical or linear baseline, with the forest modelling residuals; or a different model family.) Transition: the controls that shape a forest.

Forest controls have practical meanings

BAGGING AND RANDOM FORESTS

Forest controls have practical meanings

Parameter	What it changes	Diagnostic
n_estimators	Number of trees	Score stability and runtime
max_features	Features considered at each split	Tree diversity and fit
min_samples_leaf	Smallest terminal group	Smoothing and rare patterns
max_depth	Longest decision path	Complexity and memory

Tune leaf size and feature subsampling before expanding the search.

◆ Defaults are starting points: max_features is "sqrt" for classifiers and 1.0 for regressors.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 20

[3 min] Use a modest initial search. An arbitrarily large grid wastes time and can overfit validation noise (Module 5 covers search strategies and nested cross-validation).

n_estimators: more trees never make a random forest overfit in the usual sense; they reduce the random fluctuation of the average and cost memory and prediction time. Pick a value where the out-of-bag or validation curve has flattened. max_features: the diversity–strength dial from the previous slides. min_samples_leaf: the most useful smoothing control; larger leaves average more rows and ignore rare patterns (good for noisy labels, bad if the rare pattern is the event you care about). max_depth: an alternative complexity limit; it also bounds model size and latency, which matters for edge deployment.

Discuss class weights only when class imbalance and costs justify them. Scaling is generally unnecessary for tree splits, but other preprocessing (imputation, encoding) still matters. Transition: the out-of-bag estimate.

Each row is scored only by trees that never saw it

BAGGING AND RANDOM FORESTS

Each row is scored only by trees that never saw it

- About a third of the trees leave out any given row.
- A row's out-of-bag prediction uses only those trees.
- The OOB score needs no separate validation fold.

Trees (each fitted to one bootstrap sample)

	T1	T2	T3	T4	T5	T6	OOB vote
W1 1500.0 m	×1	×1	out	×1	out	out	T3, T5, T6
W1 1500.5 m	×1	×1	×1	×1	out	×2	T5
W1 1501.0 m	×2	×2	×1	×1	out	×1	T5
W1 1501.5 m	×1	out	×1	×2	×3	out	T2, T6
W1 1502.0 m	×2	out	×2	out	out	×2	T2, T4, T5
W1 1502.5 m	out	×1	×1	out	×2	×1	T1, T4
W1 1503.0 m	×1	×1	×2	×2	×1	×1	none
W1 1503.5 m	out	×1	×1	out	×1	×1	T1, T4
W1 1504.0 m	×2	×2	out	×1	×2	out	T3, T6
W1 1504.5 m	out	×1	×1	×2	×1	×2	T1

17 of 60 cells out of bag (28%); expected $(1 - 1/n)^n$ = 35%

◆ Neighbouring depth rows are often in-bag when a row is out of bag.

Bootstrap draws for 10 adjacent depth rows of one synthetic well and 6 trees (seed 2).

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 21

[3 min] Read one row across. The row at 1500.0 m was drawn by trees T1, T2, and T4, and left out by T3, T5, and T6; its out-of-bag (OOB) prediction is the average of T3, T5, and T6 only. The row at 1503.0 m was drawn by all six trees, so with six trees it has no OOB prediction; with hundreds of trees every row gets many OOB votes. Here 17 of 60 cells are out of bag (28%); the expected share for n = 10 is $(0.9)^{10}$ = 35%.

The OOB score compares every row's OOB prediction with its label: an internal validation estimate obtained without a separate fold. In scikit-learn set oob_score=True; the score is in oob_score_ after fitting.

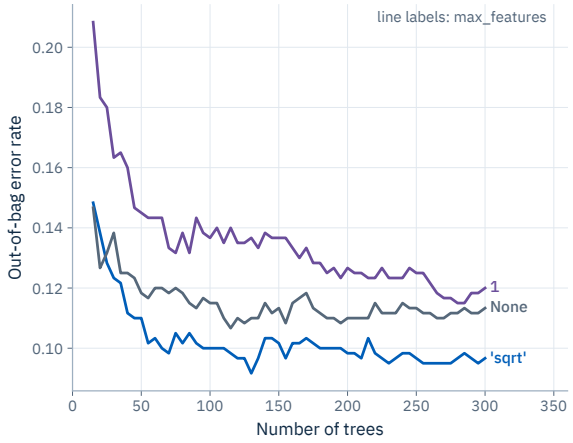
Now look at the callout. The row at 1502.5 m is out of bag for T1, but its neighbours at 1502.0 m (×2) and 1503.0 m (×1) are in-bag for T1. Tree T1 has seen almost the same logs and the same facies half a metre away. Keep that in mind for the assumptions slide. Transition: how the OOB error evolves with more trees.

Out-of-bag error settles as trees are added

BAGGING AND RANDOM FORESTS

Out-of-bag error settles as trees are added

- Error falls fast, then flattens: extra trees stabilize the average.
- Pick n_estimators on the flat part, for cost rather than accuracy.
- max_features shifts the level of the curve, not its shape.



Synthetic classification data (600 rows, 10 features, 5% label noise, seed 3); forests grown with warm_start, seed 0.

[3 min] Each curve grows one forest with warm_start=True, adding trees and recording the OOB error. All three curves drop steeply over the first 50 trees and are nearly flat after about 150. At 300 trees the OOB error is 0.097 with max_features = "sqrt" (3 of 10 features), 0.113 with all 10 features (bagging), and 0.120 with one feature per split. On this dataset "sqrt" is the best of the three; that ordering is data dependent.

Two practical uses. First, choose n_estimators where the curve is flat: beyond that point more trees buy almost nothing and cost memory and prediction time. Second, compare settings cheaply without a separate validation fold, if the rows are independent.

Common misconception: "more trees overfit". For a random forest, adding trees does not increase overfitting; the curve flattens. Overfitting comes from tree depth and leaf size, not from the number of trees. Boosting behaves differently, as Part 3 shows. Transition: the code.

A forest reports its out-of-bag score for free

BAGGING AND RANDOM FORESTS

A forest reports its out-of-bag score for free

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GroupKFold, cross_val_score

forest = RandomForestClassifier(
    n_estimators=300,        # enough for a stable average
    max_features="sqrt",     # features tried at each split
    min_samples_leaf=3,      # smallest leaf allowed
    oob_score=True, n_jobs=-1, random_state=0,
)
forest.fit(X, y)
print(f"OOB accuracy     {forest.oob_score_:.3f}")
s = cross_val_score(forest, X, y, groups=wells, cv=GroupKFold(4))
print(f"grouped accuracy {s.mean():.3f}")
```

OUTPUT

```
OOB accuracy     0.911
grouped accuracy 0.785
```

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 23

[3 min] Context: the synthetic facies logs of eight wells (X: GR, RHOB, NPHI, PE; y: facies; wells: well names). The forest is fitted once on all rows and reports its OOB accuracy, 0.911. The same settings evaluated with four grouped folds, where each fold holds out two whole wells, score 0.785.

Ask the room: which number would you report to an asset team that wants facies on the next well? The grouped one, because it matches the deployment question. The next slide explains the 13-point gap.

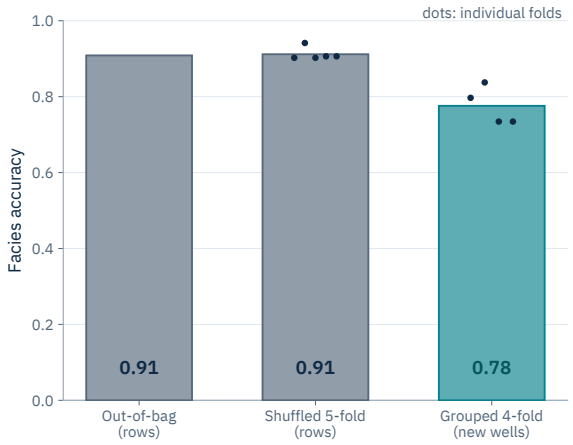
Code notes: oob_score=True requires bootstrap=True (the default for random forests). n_jobs=-1 uses all cores; do not also parallelize an outer search on the same cores. random_state makes the forest reproducible, which matters for honest comparisons. The OOB score is computed on the training rows only; it never touches a test set. Transition: the assumptions behind the OOB estimate.

Out-of-bag evaluation has assumptions

BAGGING AND RANDOM FORESTS

Out-of-bag evaluation has assumptions

- Each training row receives predictions from trees that did not sample it.
- This is an internal estimate for approximately independent rows.
- It does not protect against leakage across related well samples.
- Keep grouped or chronological evaluation for those deployment questions.



Synthetic facies logs, 8 wells with well offsets and depth-correlated noise (seed 7); forest of 300 trees, seed 0.

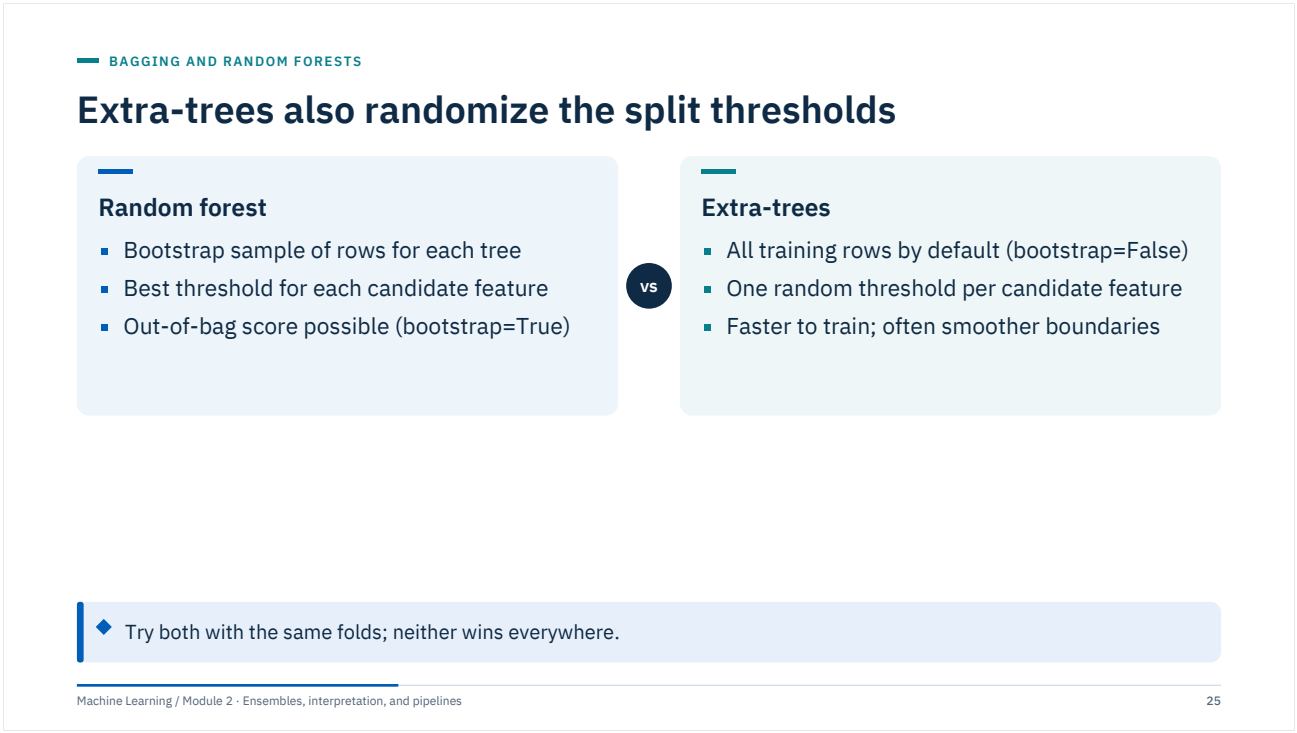
Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 24

[4 min] The bars compare three estimates for the same forest settings. Out-of-bag (0.91) and shuffled 5-fold cross-validation (0.91) agree with each other, because both hold out individual rows. Grouped 4-fold cross-validation, which holds out whole wells, gives 0.78, with folds ranging from 0.73 to 0.84.

Why the gap: a depth row can be out of bag while adjacent rows from the same well are in bag. Those neighbours share the well's calibration offset and nearly identical log values, so the forest "recognizes" the well. On a genuinely new well that information is unavailable. The OOB estimate is not wrong; it answers the question "how well does the model predict a random missing row from wells it has seen?", which is not the deployment question.

The same caution applies to time series (a row out of bag while the next hour is in bag) and to repeated tests on one piece of equipment. OOB is useful for tuning n_estimators or comparing settings on independent rows; it does not replace a deployment-matched external test. Transition: extra-trees.

Extra-trees also randomize the split thresholds



[2 min] Extremely randomized trees (Geurts, Ernst and Wehenkel, 2006) push randomization further. For each candidate feature a threshold is drawn at random between the feature's minimum and maximum in the node; the best of these random splits is kept. Because thresholds are not optimized, training is faster and individual trees are more diverse, so the ensemble's variance can drop further, usually at the cost of a little more bias per tree.

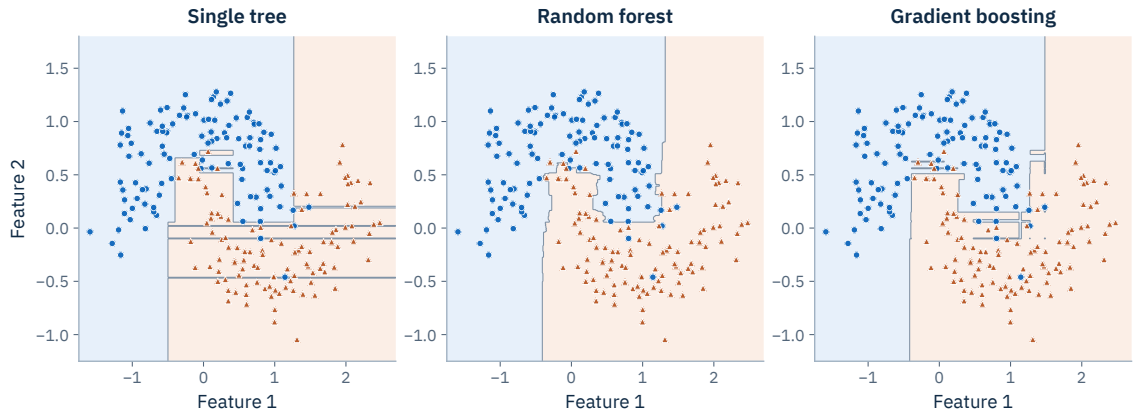
In scikit-learn: ExtraTreesClassifier and ExtraTreesRegressor, with the same controls as random forests. Their default is bootstrap=False, so every tree sees all rows and there is no out-of-bag score unless you set bootstrap=True.

When to try them: noisy tabular data where a forest overfits, or when training time matters. As with every candidate, compare on the same grouped folds, the same metric, and the same tuning budget. Transition: what the ensembles' boundaries look like.

Ensembles change decision boundaries

BAGGING AND RANDOM FORESTS

Ensembles change decision boundaries



Tree, forest, and boosting fits on the same synthetic data. Boundary appearance alone does not establish generalization quality.

[2 min] Compare isolated regions and smoothness. The single tree carves thin rectangles around individual points. The forest (100 trees, min_samples_leaf = 4) produces a boundary built from many averaged rectangles; boosting (70 depth-2 trees) builds its boundary additively from small corrections, which Part 3 explains.

Ask learners to identify evidence that is missing: held-out performance, subgroup errors, and training stability. A pleasing boundary on training points is not evidence of generalization, and a boundary that fits every training point is usually a warning sign.

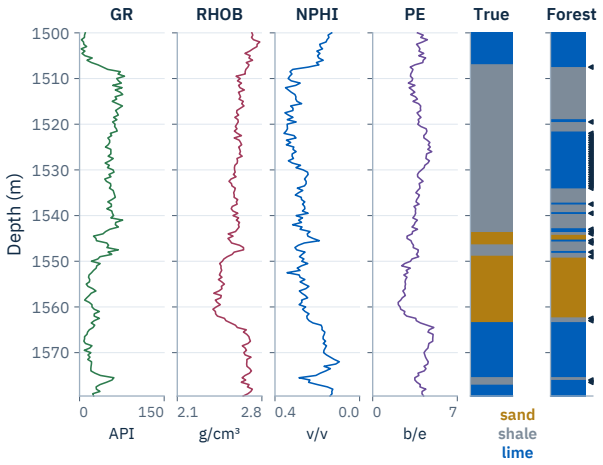
Reuse the experimental conditions, not an assumed winner. In Lab 1 the three model types are compared on identical folds so that differences can be attributed to the model rather than to the split. Transition: an energy case with a held-out well.

A forest classifies facies on a held-out well

BAGGING AND RANDOM FORESTS

A forest classifies facies on a held-out well

- Trained on wells W1–W7, scored on W8 only.
- 75% of W8's rows are correct.
- One 12 m shale interval is called limestone; most other errors sit at bed boundaries.



Synthetic logs; forest of 300 trees (min_samples_leaf 3, seed 0).

◆ Report errors per well and per facies, not only overall.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 27

[4 min] Walk the log display from left to right: gamma ray, bulk density, neutron porosity, photoelectric factor, the true facies, and the forest's prediction; arrows mark misclassified rows. The forest was trained on seven wells and never saw W8.

Accuracy on W8 is 0.75 (120 of 160 rows). Per facies: limestone recall 1.00, sandstone 0.82, shale 0.58. Most errors come from one interval, 1522–1534 m, where shale is predicted as limestone: in W8 the shale reads low in gamma ray, dense, and with high PE, much like the limestones of the training wells. It is a well-level offset of the kind that calibration differences produce in real wells. Of the 15 remaining errors, 12 are single rows within a metre of a bed boundary, where logs average two lithologies; the other three are isolated shale rows called limestone in the same shale bed.

Discussion prompt: what would you do next? Good answers: check whether W8's logs need normalization to a reference well; add a baseline (for example PE and GR cut-offs); report per-well results from grouped cross-validation, not one well; ask a petrophysicist whether that interval is a calcareous shale. Data are synthetic, so no field claim is made. Transition: a check exercise.

Which forest change would you try first?

PRACTICE

5 min Pairs

Which forest change would you try first?

Scenario: a forest predicts ESP trips from sensor summaries of 12 wells. Training accuracy 1.00, OOB 0.93, grouped CV by well 0.79.

1. Grow 2,000 trees instead of 300: what changes?
2. Raise min_samples_leaf from 1 to 20: what might improve, and how do you check?
3. Switch max_features from "sqrt" to all features: likely effect?
4. Add the well identifier as a feature: acceptable?

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 28

[5 min] The numbers are a hypothetical scenario, not a result. Answers:

(1) More trees make the average more stable and slow everything down, but they will not close the gap between 0.93 and 0.79; the OOB curve is almost certainly flat by 300 trees. (2) Larger leaves smooth the model and stop it memorizing well-specific patterns; this is the most promising change. Check it with the grouped cross-validation, not with OOB, and look at per-well results. (3) Using all features increases tree correlation (plain bagging); it rarely helps a forest that already overfits. (4) No. A well identifier lets the model memorize each training well and is meaningless for a new well; grouped CV would expose it, and OOB would reward it. Wells can be used for grouping, never as predictors for new-well deployment.

Common error: treating the 1.00 training accuracy as a problem to fix; unpruned forests always fit the training set almost perfectly, and that is not by itself a sign of trouble. The gap between OOB and grouped scores is the real signal. Transition: break, then boosting.

From AdaBoost to gradient boosting



Part 3 [55 min]. Boosting builds an ensemble sequentially. Each new model is trained on a version of the problem that emphasizes what the current ensemble gets wrong: re-weighted rows in AdaBoost (Freund and Schapire, 1997), residuals or negative gradients in gradient boosting (Friedman, 2001).

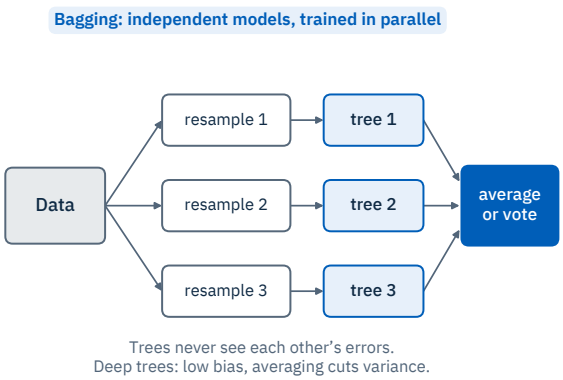
The practical difference from forests is that boosting can overfit as stages are added, so the number of stages, the learning rate, and early stopping become the central controls. By the end of this part learners should be able to perform one boosting update by hand, write down the pseudo-residual for four common losses, and read a validation curve to decide where to stop.

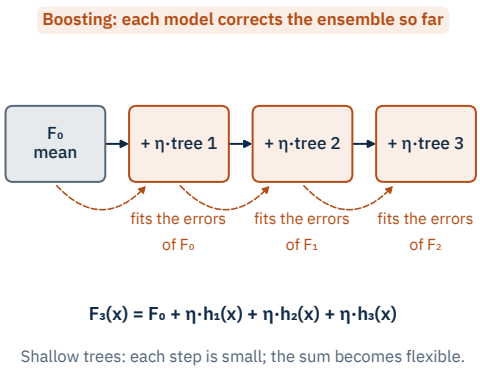
Ask before starting: in a forest, does tree 200 know anything about tree 199? (No; they are independent given the data.) In boosting, tree 200 exists only because of trees 1 to 199. Transition: the two training logics side by side.

Bagging runs in parallel; boosting runs in sequence

FROM ADABOOST TO GRADIENT BOOSTING

Bagging runs in parallel; boosting runs in sequence





Schematic comparison of bagging and boosting.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 30

[3 min] Left: bagging trains every tree independently on its own resample; trees never see each other's errors and can be trained in parallel on separate cores. Deep trees have low bias and high variance, and averaging removes much of the variance.

Right: boosting starts from a constant prediction F_0 (the mean for squared loss) and adds one small tree at a time. Each tree is fitted to the errors of the current model, and only a fraction η of its prediction is added. Shallow trees are weak on their own, but the sum of hundreds of small corrections becomes a flexible function.

Consequences to draw out: boosting stages cannot be trained in parallel (the libraries parallelize within a tree instead); boosting can keep reducing training error until it fits noise, so it needs a stopping rule; forests are harder to overfit by adding trees. Avoid the oversimplification that bagging only reduces variance while boosting only reduces bias; both affect generalization in several ways. Transition: the comparison table.

Bagging and boosting use different training logic

FROM ADABOOST TO GRADIENT BOOSTING

Bagging and boosting use different training logic

Question	Bagging	Gradient boosting
How are models trained?	Separately on perturbed samples	Sequentially against current loss
What combines them?	Average or vote	Additive weighted updates
Typical base learner	Relatively deep tree	Shallow tree
Main sensitivity	Correlated trees, leaf size	Learning rate, stages, noisy targets

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 31

[2 min] Use the table as a summary of the diagram. Avoid the oversimplification that bagging only reduces variance while boosting only reduces bias. Both can affect generalization in several ways. The important distinction is the training dependency and how each method controls complexity.

Row by row: bagging perturbs the data and averages; boosting changes the target of each new learner and adds. Bagging uses deep trees because each must be a good, low-bias predictor on its own; boosting uses shallow trees (depth 2 to 8, or a limited number of leaves) because each only needs to capture part of the remaining structure. Bagging's main failure mode is correlated trees; boosting's is fitting noise when there are too many stages, too large a learning rate, or noisy targets such as mislabelled events.

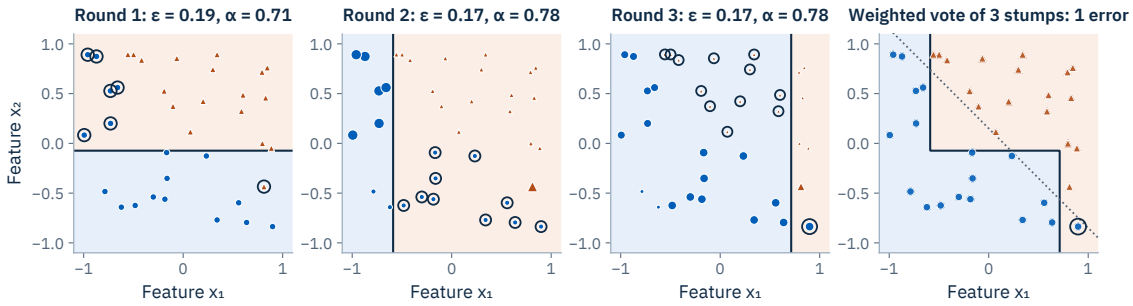
Ask: which would you expect to suffer more from a few badly mislabelled rows? (Boosting, especially AdaBoost, which keeps increasing their weight.) Transition: AdaBoost in pictures.

AdaBoost up-weights the rows it gets wrong

FROM ADABOOST TO GRADIENT BOOSTING

AdaBoost up-weights the rows it gets wrong

Marker area = row weight; rings mark rows the current stump gets wrong, which gain weight next round.



Discrete AdaBoost with decision stumps on 36 synthetic points (seed 12); dotted line: true boundary $x_1 + x_2 = 0.15$.

[4 min] Round 1: all 36 rows have weight 1/36. The best stump splits on x_2 and misclassifies 7 rows, so its weighted error is $\varepsilon = 7/36 = 0.19$. Round 2: the misclassified rows now carry larger markers and the next stump, a split on x_1, is chosen to get them right; it misclassifies 10 different rows, weighted error 0.17. Round 3 shifts attention again.

Last panel: the weighted vote of the three stumps is a staircase that approximates the diagonal boundary; only one training row remains misclassified. No single stump could do this, because each can only draw one axis-parallel line.

Point out the risk: a mislabelled row is misclassified by almost every stump, so its weight keeps growing and later stumps contort themselves around it. That is why AdaBoost is sensitive to label noise, and why gradient boosting with robust losses is usually preferred for noisy operational data. Transition: how much say each stump gets.

AdaBoost gives accurate stumps a larger say

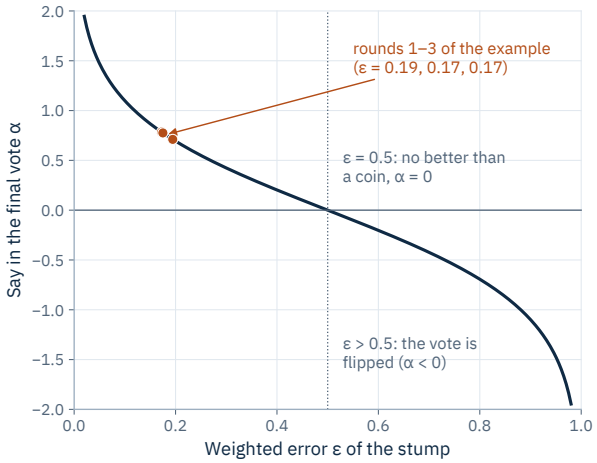
FROM ADABOOST TO GRADIENT BOOSTING

AdaBoost gives accurate stumps a larger say

$$\alpha_m = \tfrac{1}{2}\ln\frac{1-\varepsilon_m}{\varepsilon_m}$$

$$F(x) = \operatorname{sign}\left(\sum_m \alpha_m\, h_m(x)\right)$$

- ε_m: weighted error of stump m; weights sum to 1.
- Misclassified rows are multiplied by e^{α}, correct rows by $e^{-\alpha}$; then renormalize.
- After each update, the misclassified rows hold exactly half the weight.



Exact formula; points are the three rounds of the example.

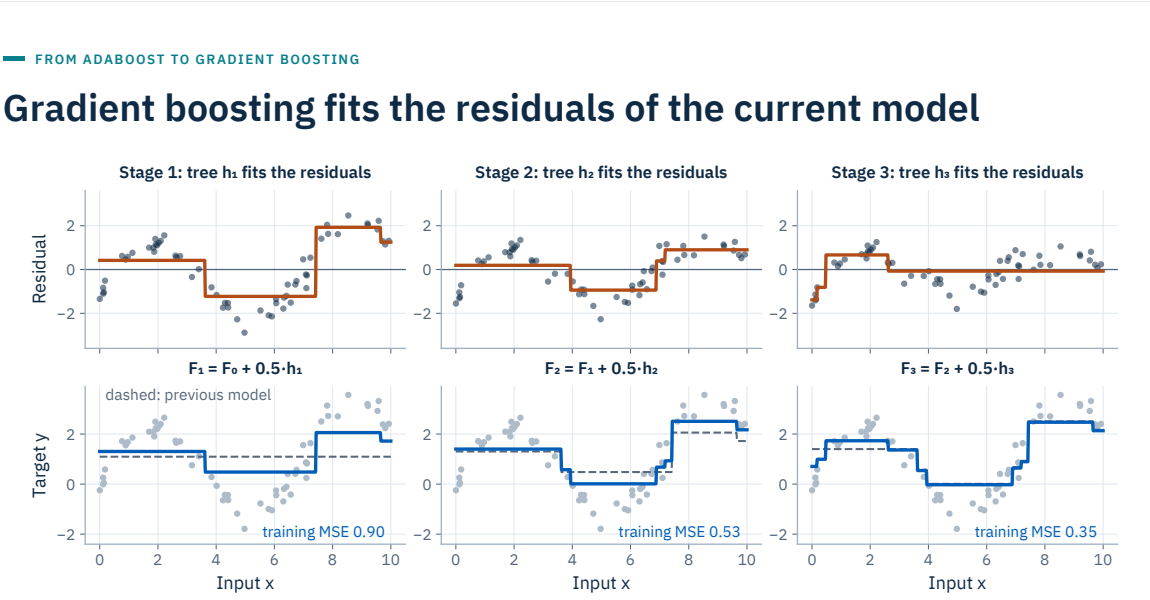
Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 33

[3 min] Here the labels are coded as h(x) = −1 or +1. The say α of a stump depends only on its weighted error ε. The round-1 stump has ε = 7/36 = 0.19 and gets α = ½ ln(29/7) = ½ ln(4.14) = 0.71. A stump no better than a coin (ε = 0.5) gets α = 0, and one that is worse than a coin would get a negative say, flipping its vote.

Weight update in numbers for round 1: seven misclassified rows go from 1/36 = 0.0278 to 0.0714 each (×2.57 after renormalizing); the 29 correct rows go to 0.0172 (×0.62). The misclassified rows now hold 7 × 0.0714 = 0.50 of the total weight, which is always the case: the next stump sees the previous one's mistakes and successes as equally important in total.

scikit-learn: AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=1), n_estimators=..., learning_rate=...); in recent versions only this discrete (SAMME) variant remains. Transition: gradient boosting generalizes the idea to any differentiable loss.

Gradient boosting fits the residuals of the current model



Synthetic data (n = 60, seed 21); F_0 = mean of y; depth-2 regression trees; learning rate 0.5. Top: residuals and the tree fitted to them. Bottom: updated model.

[4 min] Read the columns top to bottom. Stage 1: the model is the constant F_0 (the mean). The residuals $y - F_0$ show clear structure; a depth-2 tree h_1 (four leaves, orange) summarizes that structure. The new model $F_1 = F_0 + 0.5 \cdot h_1$ moves halfway towards the tree's correction. Training MSE falls to 0.90.

Stage 2: residuals are computed from F_1, not from the original targets; they are smaller and the new tree targets what is left. MSE 0.53. Stage 3: MSE 0.35. Each tree on its own is a crude four-step function; the sum of many such functions becomes flexible.

Two ideas to stress. First, every tree is fitted to the current residuals, so the trees are strongly dependent. Second, the learning rate $\eta = 0.5$ deliberately takes only part of each correction; smaller steps need more trees but usually generalize better. Ask: what would happen with $\eta = 1$ and deep trees? (The first tree would fit the noise almost immediately.) Transition: why "gradient".

Boosting is gradient descent on the predictions

FROM ADABOOST TO GRADIENT BOOSTING

Boosting is gradient descent on the predictions

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$
$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right] F = F_{m-1}$$

- Start with a constant prediction that minimizes the loss.
- Fit the next tree h_m to the negative gradients r_{im} (pseudo-residuals).
- Shrink each update by the learning rate η.
- Squared loss: r_{im} is the ordinary residual $y_i - F_{m-1}(x_i)$.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 35

[3 min] Connect to gradient descent from Module 1: there we moved parameters w against the gradient of the loss. Here the "parameters" are the model's predictions at the training rows. The negative gradient of the loss with respect to each prediction says which direction and how much each prediction should move. A regression tree fitted to these values generalizes the step to new inputs x, where no gradient is available.

For squared loss L = ½(y − F)², the derivative is −(y − F), so the negative gradient is the ordinary residual: gradient boosting with squared loss is residual fitting. For other losses the tree fits pseudo-residuals, which are not raw residuals. Some implementations then re-estimate each leaf's value with a line search or a Newton step (the histogram libraries use second-order information).

Define symbols: F_m is the model after m stages, h_m the m-th tree, η the learning rate (shrinkage), L the loss, i indexes training rows. Transition: an update by hand.

A boosting update by hand

PRACTICE

5 min Individual

A boosting update by hand

Observed targets: [2, 4, 6]. Current predictions: [4, 4, 4].

The new tree predicts residuals [−2, 0, 2]. Learning rate = 0.1.

1. Calculate the updated predictions.
2. Compare mean squared error before and after the update.

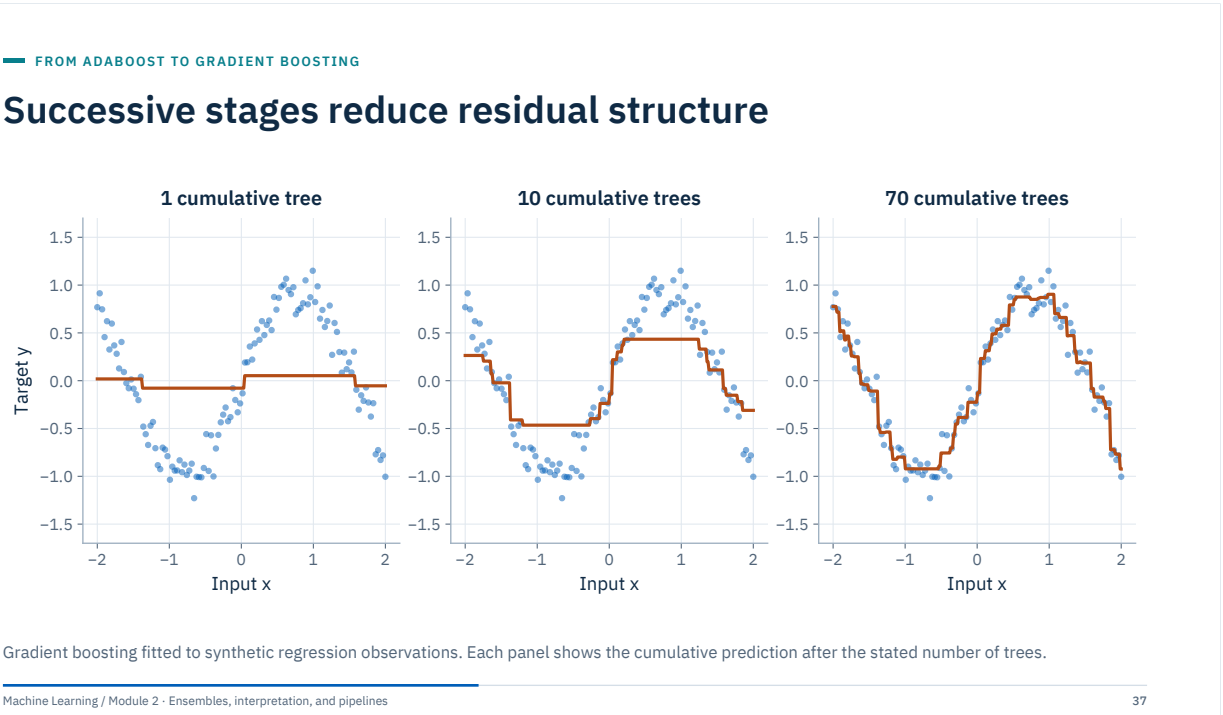
[5 min] Expected predictions are [3.8, 4, 4.2]: each prediction moves by 0.1 × the tree's output, so 4 + 0.1·(−2) = 3.8, 4 + 0 = 4, and 4 + 0.1·2 = 4.2.

Old MSE: residuals [−2, 0, 2], squared [4, 0, 4], mean 8/3 ≈ 2.667. New MSE: residuals [2 − 3.8, 0, 6 − 4.2] = [−1.8, 0, 1.8], squared [3.24, 0, 3.24], sum 6.48, mean 6.48/3 = 2.16.

The tree here predicts the residuals exactly, yet the update removes only part of the error because η = 0.1. Ten such steps would leave residuals of $2 \times 0.9^{10} \approx 0.70$. This is an arithmetic illustration with a specified tree, not a claim that a shallow tree can fit all possible residual patterns.

Common errors: adding the full tree output (forgetting η), or computing new residuals from the original predictions. Ask: why accept slower progress? Small steps make each tree's mistakes less costly, which acts as regularization. Transition: the same process on real curves.

Successive stages reduce residual structure



[2 min] Emphasize cumulative prediction versus the prediction from one newly added tree. After one tree (learning rate 0.1, depth 2) the model is almost flat: a single shrunken tree barely moves the constant start. After ten trees the broad shape of the sine wave appears; after seventy the staircase follows the data closely.

The early constant fit becomes a piecewise approximation. Eventually additional stages may fit noise, so inspect held-out learning curves, which we do on the next slides after looking at other losses.

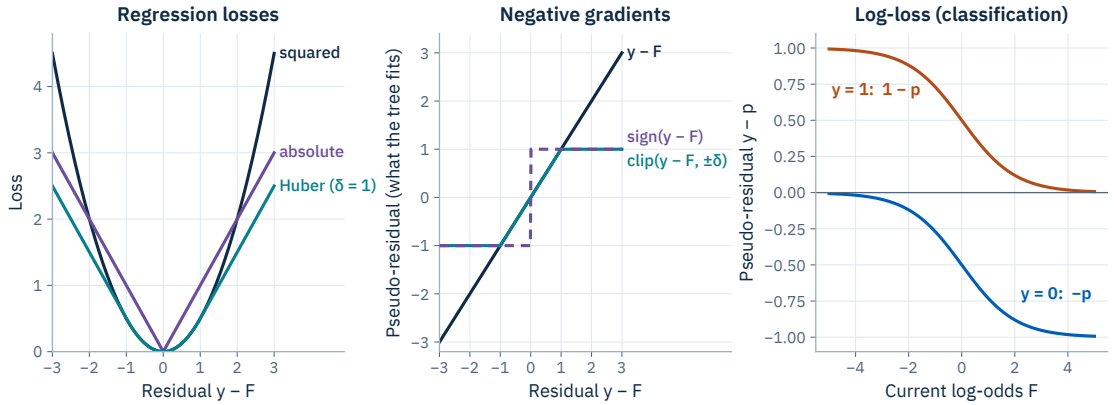
Ask: at seventy trees, is the model already following individual noisy points in some places? (Look near the peaks.) That is the first sign that more stages would begin to fit noise. Transition: what changes when the loss is not squared error.

The loss decides what each tree fits

FROM ADABOOST TO GRADIENT BOOSTING

The loss decides what each tree fits

Each tree fits the negative gradient of the loss at the current predictions: the pseudo-residual.



Exact functions. Huber threshold δ = 1; p = 1 / (1 + e^(−F)).

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 38

[4 min] Left: three regression losses as functions of the residual y − F. Squared loss grows quadratically, so a residual of 3 costs 4.5 and dominates the fit. Absolute loss grows linearly. Huber loss is quadratic near zero and linear beyond δ.

Middle: the corresponding pseudo-residuals. Squared loss hands the tree the raw residual, so one bad reading (a spike in ROP from a sensor glitch) pulls the next tree strongly. Absolute loss hands the tree only the sign, so every row pushes with the same force. Huber passes small residuals unchanged and clips large ones at ±δ: a compromise for mostly well-behaved data with occasional bad readings.

Right: binary classification with log-loss. The model works on log-odds F, and p = 1/(1 + e^(−F)). The pseudo-residual is y − p: a confident correct row (p near y) contributes almost nothing; a confident wrong row contributes almost ±1. This is gradient boosting's version of "focus on the mistakes", smoother than AdaBoost's weights. Transition: the summary table.

Each loss has its own pseudo-residual

FROM ADABOOST TO GRADIENT BOOSTING

Each loss has its own pseudo-residual

Loss	L(y, F)	Pseudo-residual	Use when
Squared	½(y − F)²	y − F	Default for regression
Absolute	abs(y − F)	sign(y − F)	Heavy-tailed target errors
Huber (δ)	Quadratic within ±δ, linear beyond	y − F clipped to ±δ	Mostly clean, some bad readings
Log-loss	−y ln p − (1 − y) ln(1 − p)	y − p	Binary classification (F = log-odds)

In scikit-learn: loss="squared_error", "absolute_error", "huber", or "log_loss".

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 39

[2 min] The table summarizes the previous figure. Stress that the loss should match the decision: if large ROP errors matter much more than small ones, squared loss is appropriate; if the target contains occasional recording errors that should not dominate, Huber or absolute loss is safer; if you need a median rather than a mean prediction, absolute loss estimates the conditional median.

scikit-learn details (1.7): GradientBoostingRegressor accepts "squared_error", "absolute_error", "huber", and "quantile"; for Huber the alpha parameter (default 0.9) sets δ adaptively as that quantile of the absolute residuals. HistGradientBoostingRegressor offers "squared_error", "absolute_error", "gamma", "poisson", and "quantile", but not Huber. Classifiers use "log_loss"; GradientBoostingClassifier also offers "exponential", which recovers AdaBoost.

Quantile loss deserves a mention for energy work: fitting the 10th and 90th percentiles of ROP or production gives a prediction interval; NN Day 5 (Sequence models and attention) develops quantile forecasts further. Transition: compute some pseudo-residuals.

Pseudo-residuals by hand

PRACTICE

5 min Pairs

Pseudo-residuals by hand

Current residuals y − F for four rows: −3, −0.5, 0.2, and 4.

1. Write the pseudo-residuals the next tree fits under squared, absolute, and Huber (δ = 1) loss.
2. Which loss lets the row with residual 4 dominate the next tree?
3. Log-loss: y = 1 with p = 0.8, and y = 0 with p = 0.3. What are the pseudo-residuals?

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 40

[5 min] Answers.

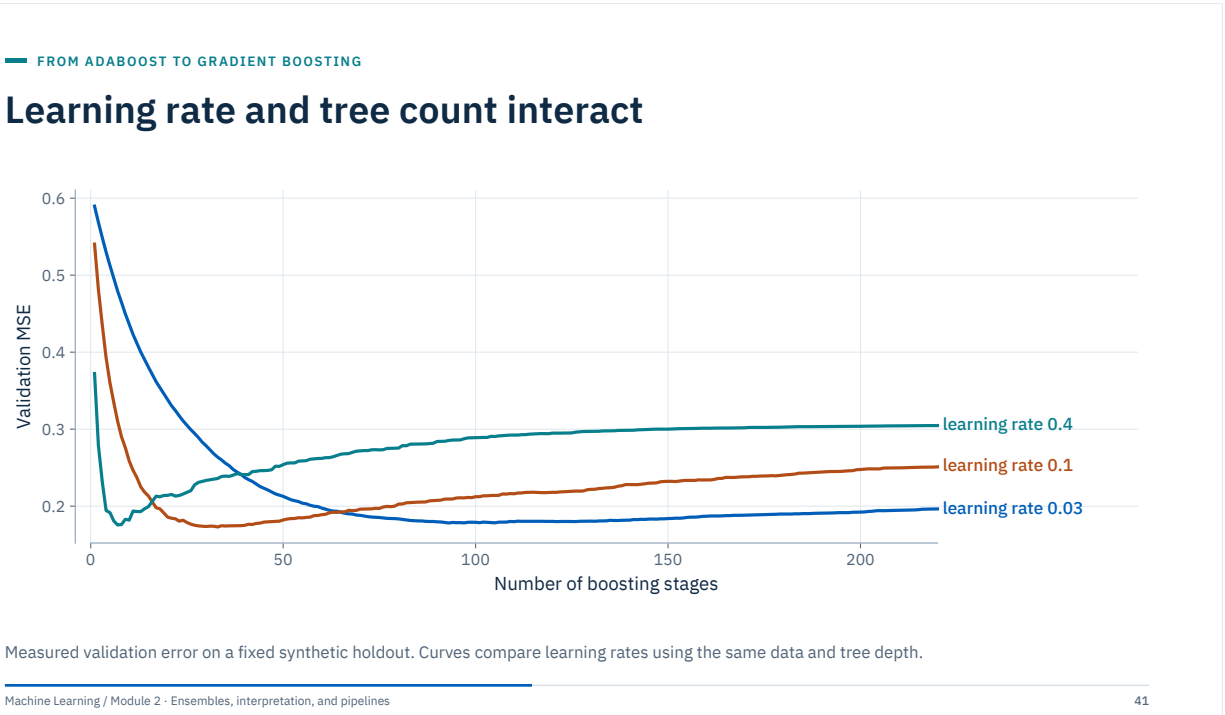
(1) Squared loss: the residuals themselves, [−3, −0.5, 0.2, 4]. Absolute loss: the signs, [−1, −1, 1, 1]. Huber with δ = 1: small residuals pass, large ones are clipped: [−1, −0.5, 0.2, 1].

(2) Squared loss: the row with residual 4 has a pseudo-residual of 4, eight times the second row's magnitude, so a regression tree fitted to squared errors bends towards it. Under absolute and Huber loss it counts no more than a residual of 1.

(3) y − p: 1 − 0.8 = 0.2 and 0 − 0.3 = −0.3. The second row is wrong with moderate confidence and pushes harder than the first, which is right with 0.8 confidence.

Common errors: applying the sign to the target rather than to the residual; clipping Huber at δ for the loss value but forgetting that the gradient is clipped at ±δ. Ask: which loss would you choose for ROP when the logging system sometimes records a spike during connections? (Huber or absolute.) Transition: learning rate and number of trees.

Learning rate and tree count interact



[2 min] Ask where each curve stops improving. With learning rate 0.4 the validation error reaches its minimum within about ten stages and then rises steadily as later trees fit noise. With 0.1 the minimum comes around thirty stages. With 0.03 the curve descends slowly, bottoms out after about ninety stages, and stays flat for longer.

A lower learning rate usually requires more stages but gives a wider, flatter optimum, which makes the choice of the number of stages less critical. There is no guarantee of a better final model: on this dataset the minima of the three curves are similar.

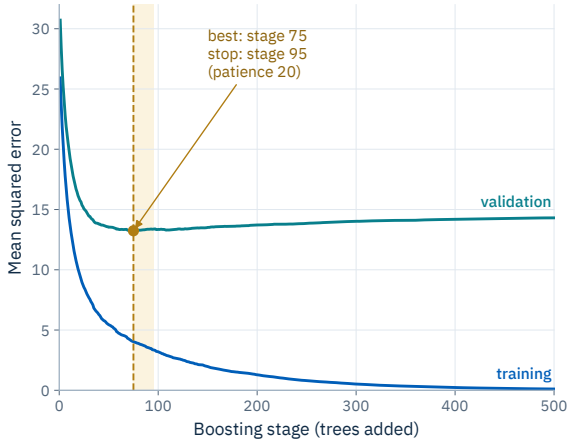
Practical rule: fix a moderately small learning rate (for example 0.05 to 0.1), then let early stopping on validation data choose the number of stages. Early stopping must use validation data separate from the final test. Transition: early stopping in detail.

Early stopping chooses the number of trees

FROM ADABOOST TO GRADIENT BOOSTING

Early stopping chooses the number of trees

- Hold out validation rows: another well or a later period.
- Stop after a patience of N stages without improvement.
- Keep the final test rows out of this loop.



◆ Training error always falls; only validation error can say stop.

Friedman #1 synthetic regression: 400 training and 2,000 validation rows; learning rate 0.1, depth 3. Stop rule: 20 stages without improvement.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 42

[4 min] The blue training error keeps falling towards zero as stages are added. The teal validation error falls until stage 75 (MSE 13.2), then creeps up to 14.3 by stage 500. A patience rule stops training once 20 stages pass without improvement, here at stage 95, and keeps the model near the minimum.

How the libraries do it. scikit-learn's GradientBoostingRegressor stops when n_iter_no_change is set, using a random validation_fraction of the training rows; HistGradientBoosting uses early_stopping, validation_fraction, and n_iter_no_change, and from version 1.7 also accepts an explicit X_val and y_val. XGBoost and LightGBM take an evaluation set and a number of early-stopping rounds.

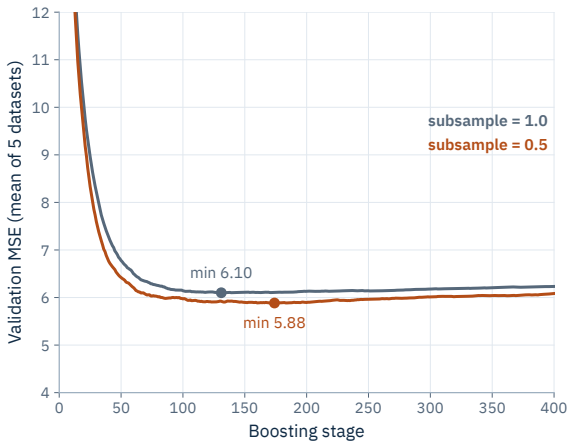
Energy caution: a random validation fraction of rows has the same problem as out-of-bag scores. If rows come from wells or time, pass a validation set of whole wells or a later period, and keep the final test set outside this loop entirely: the stopping point is a tuned hyperparameter. Transition: other regularizers.

Regularization in boosting

FROM ADABOOST TO GRADIENT BOOSTING

Regularization in boosting

- Limit tree depth or the number of leaves.
- Use a minimum leaf size to reduce fragile local updates.
- Row or feature subsampling reduces dependence on individual samples.
- Tune learning rate and iteration count together.



Friedman #1 synthetic regression (noise sd 2), mean of five datasets; learning rate 0.1, depth 3.

[3 min] The figure shows stochastic gradient boosting (Friedman, 2002): each tree is fitted to a random half of the training rows (subsample = 0.5). Averaged over five synthetic datasets, the minimum validation MSE is 5.88 with subsampling against 6.10 without, a small but consistent improvement here. Subsampling also makes each stage faster. It is not guaranteed to help; treat it as a setting to validate.

Distinguish sampling fractions from the train–validation split. Sampling happens within training data at every stage; it has nothing to do with evaluation. Model-specific implementations add further penalties (for example L2 regularization of leaf values in XGBoost, LightGBM, and HistGradientBoosting via l2_regularization) and constraints. Avoid assuming that a parameter called alpha or lambda has the same meaning across libraries: check each library's documentation.

A good order for tuning: learning rate fixed and early stopping on; then depth or leaves; then minimum leaf size; then subsampling. Transition: the settings in code.

Three controls set a gradient-boosting run

FROM ADABOOST TO GRADIENT BOOSTING

Three controls set a gradient-boosting run

```
gbr = GradientBoostingRegressor(
    loss="huber",           # robust to bad readings
    learning_rate=0.05,     # shrinkage eta
    n_estimators=2000,      # upper limit only
    max_depth=3,            # shallow trees
    subsample=0.7,          # stochastic boosting
    validation_fraction=0.15,
    n_iter_no_change=30,
    random_state=0,
)
gbr.fit(X_train, y_train)
print("stages kept:", gbr.n_estimators_)
```

OUTPUT

```
stages kept: 173
```

- Learning rate, tree size, and stages are the core trio.
- n_estimators is a ceiling; early stopping picks the count.
- The internal validation split is random rows.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 44

[3 min] Context: the synthetic drilling data, training wells D1–D5, features WOB, RPM, FLOW, BIT_HOURS, and FORMATION, target ROP in m/h. With early stopping enabled the model kept 173 of a possible 2,000 stages; the fitted count is in n_estimators_. On held-out well D6 this model's mean absolute error is 2.57 m/h.

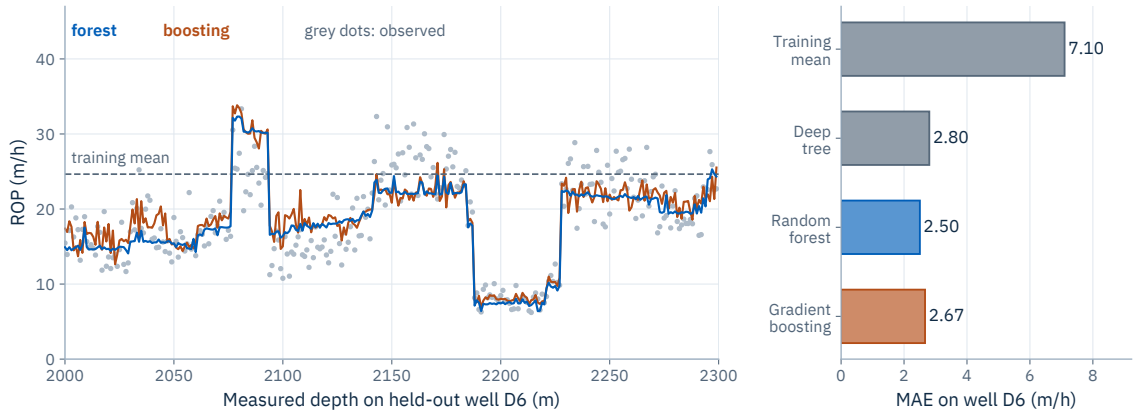
Point to the three core controls: the learning rate η, the tree size (max_depth or max_leaf_nodes), and the number of stages. They interact: a smaller learning rate needs more stages; deeper trees need fewer stages and overfit sooner.

The caution from the previous slide applies: validation_fraction = 0.15 draws random rows, and rows from the same well sit on both sides, so the stopping point may be later than a new well would justify. For grouped data, write a short loop with staged_predict on a validation well, or use HistGradientBoosting's X_val and y_val. Transition: the ROP case.

Forest and boosting score alike on a new well

FROM ADABOOST TO GRADIENT BOOSTING

Forest and boosting score alike on a new well



Synthetic drilling wells (seed 11): trained on D1–D5, evaluated on held-out well D6. Forest: 300 trees, min_samples_leaf 3; boosting: HistGradientBoostingRegressor defaults.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 45

[4 min] Left: predictions along well D6 from the random forest (blue) and histogram gradient boosting (orange), with observed ROP as grey dots and the training mean as a dashed line. Both models follow the formation changes (for example the slow interval at 2190–2230 m) and both miss some of the scatter within intervals, which comes from settings and noise not fully captured by the inputs.

Right: mean absolute error on D6. Predicting the training mean gives 7.10 m/h; a single deep tree 2.80; the forest 2.50; boosting 2.67. On this synthetic well the two ensembles are within 0.2 m/h of each other, and one well is far too little evidence to rank them.

Message: boosting is not automatically better than a forest. Its advantage usually appears with careful tuning and larger datasets; its risks are more tuning effort and sensitivity to noisy targets. To decide, compare both on several held-out wells with the same folds and report the spread, as Lab 1 does. Transition: morning recap.

Morning takeaways before the laboratory

KEY TAKEAWAYS

Morning takeaways before the laboratory

- ✓ Combining helps when members are useful and their errors differ.
- ✓ Bagging averages deep trees; random forests also decorrelate them with max_features.
- ✓ Out-of-bag scores assume unrelated rows; use grouped validation for new wells.
- ✓ Boosting adds shallow trees fitted to pseudo-residuals, shrunk by the learning rate.
- ✓ Early stopping on deployment-like validation data sets the number of stages.

[2 min] Read the five statements and ask the room which one they would find hardest to explain to a colleague. Usually it is the pseudo-residual; if so, restate: "each new tree fits the direction in which every prediction should move to reduce the loss; for squared error that is the residual".

Bridge to the lab: Lab 1 compares a single tree, a random forest, and gradient boosting with cross-validation on the training rows only, using identical folds, and then scores the sealed test rows once. The small Iris dataset keeps runs fast. Ask learners to watch for one thing in particular: whether the differences between models are larger than the differences between folds.

Logistics: the notebook is exercises/Ensemble_Methods_Student.ipynb; a solution notebook exists for the instructor. Transition: the lab brief.

Laboratory 1: tree, forest, and boosting compared

LABORATORY

60 min

Laboratory 1: tree, forest, and boosting compared

- ☐ Parts 1–2: stratified 80/20 split (random_state 42), then a single decision tree.
- ☐ Parts 3–4: a random forest and gradient boosting on the same split.
- ☐ Part 5: 5-fold CV of all three on the training rows; then open the test rows once.
- ☐ Extension: set oob_score=True and plot a staged_predict validation curve.

NOTEBOOK

`exercises/Ensemble_Methods_Student.ipynb`

EVIDENCE TO SUBMIT

- A score table: CV mean ± sd per model
- One OOB or validation-curve plot
- One sentence: is any difference larger than fold noise?

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 47

Timing [60 min]: 5 min brief, 45 min work, 10 min debrief.

Brief: the notebook uses Iris (150 rows, 4 features, 3 classes), so every fit takes seconds. Parts 1–4 make a stratified 80/20 split, print the label mapping, and fit each model once on the 120 training rows; they report only training accuracy (1.000 for all three), which cannot rank the models. Part 5 cross-validates all three on the training rows with the same five folds, and only then opens the 30 test rows, once, in a clearly labelled final-evaluation cell (section 5.2).

Extension for fast groups (section 5.1, training rows only): fit RandomForestClassifier(oob_score=True) and compare oob_score_ with the CV mean; split X_train into training and validation rows and plot validation accuracy from GradientBoostingClassifier.staged_predict against the number of stages.

Common problems: forgetting stratify=y; passing X and y instead of X_train and y_train to cross_val_score, which puts the test rows inside the folds; comparing models on different splits; reading a one-row difference on 30 test rows as a real ranking. Circulate and ask each pair what one test row is worth in accuracy (1/30 = 3.3 points).

Lab 1 debrief: the ranking is within the noise

DISCUSSION

Lab 1 debrief: the ranking is within the noise

Model	Test accuracy (30 rows)	5-fold CV on training rows
Single tree	0.933	0.942 ± 0.020
Random forest	0.900	0.950 ± 0.017
Gradient boosting	0.967	0.967 ± 0.017

- One test row is worth 3.3 points: the test ranking rests on one or two rows.
- The forest's OOB accuracy on the training rows is 0.942.

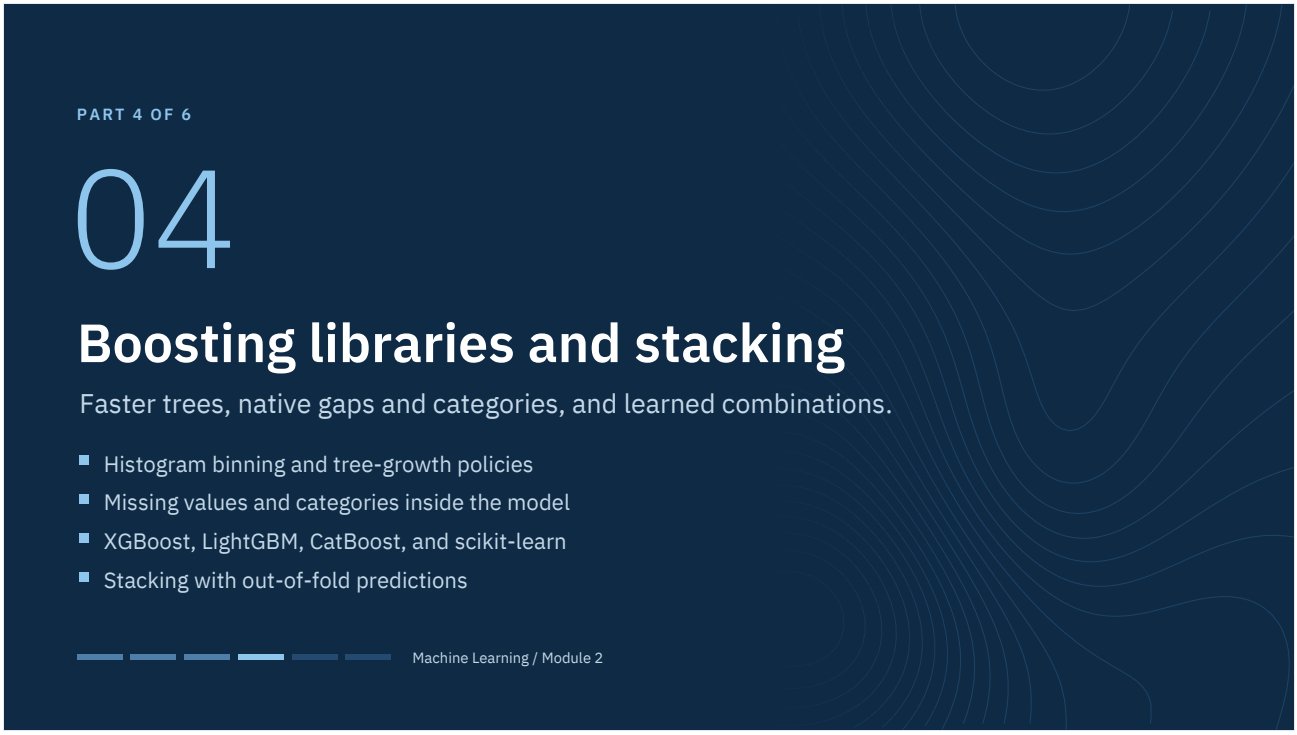
[10 min] Values are from the solution notebook run with scikit-learn 1.9.1 and the notebook's seeds (scikit-learn 1.7.2 gives the same values); small differences across other versions are possible.

Ask two or three pairs to share their tables first. Then point out that the test ranking (boosting 29/30, tree 28/30, forest 27/30) differs from the CV ranking, and that every difference is one or two test rows. With fold standard deviations around 0.02, none of the models is clearly better on Iris. Iris is small and nearly separable, so ensembles have little variance to remove; the energy examples in the lectures show where they matter.

If a pair cross-validated on all 150 rows instead, they obtained 0.953 ± 0.034 (tree), 0.967 ± 0.021 (forest), and 0.960 ± 0.033 (boosting): numbers that already include the test rows, so they cannot also be reported as independent test evidence.

Discussion: what would convince you that boosting is better? More data, repeated or grouped CV, paired comparisons on the same folds, and a practically meaningful difference. Close with the rule for the afternoon: the model choice is made on validation evidence; the test set is opened once. Transition: lunch.

Boosting libraries and stacking



Part 4 [30 min]. After lunch we look at why modern gradient-boosting libraries are fast and what their defaults mean for practitioners. Three ideas explain most of the differences: histogram binning of features, the order in which a tree is grown, and native handling of missing values and categorical inputs.

We then compare four libraries qualitatively. There is no universal speed or accuracy ranking; the right choice depends on data size, feature types, deployment constraints, and what your team can maintain. The part ends with stacking, the most flexible way to combine different models, and its leakage trap.

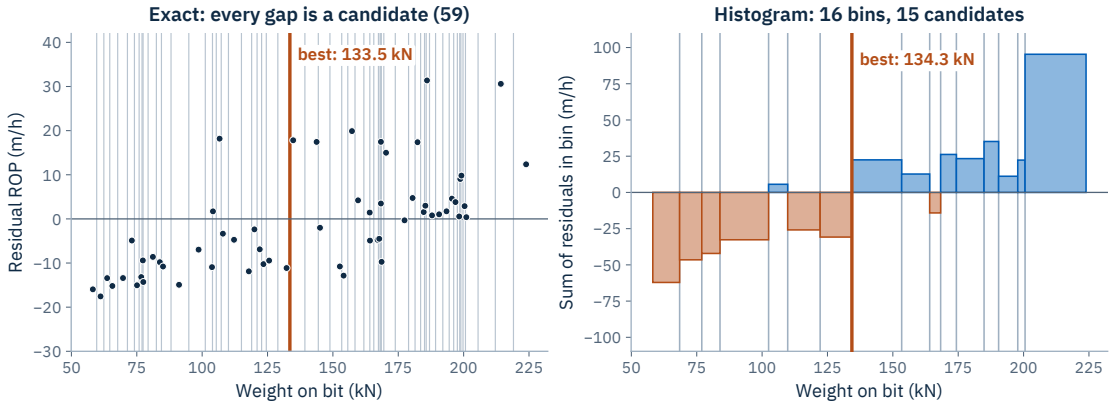
Energy framing: drilling and production tables routinely have gaps (sensor dropouts) and categorical columns (formation, equipment type, field), which is exactly where these library features matter. Transition: histogram binning.

Histogram binning makes split search cheap

BOOSTING LIBRARIES AND STACKING

Histogram binning makes split search cheap

Bin each feature once; every split then needs only per-bin sums of gradients and counts.



Sixty rows of synthetic drilling data (seed 4); residuals after the first-stage mean prediction; 16 quantile bins. Both searches choose the same partition.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 50

[4 min] Left: an exact tree search sorts the rows by WOB and considers a threshold in every gap between neighbouring values: 59 candidates for 60 rows, and millions for large tables at every node of every tree. The best split (orange) is at 133.5 kN.

Right: histogram-based boosting first maps each feature into a small number of quantile bins (16 here, 255 by default in scikit-learn and LightGBM). For each node it only needs the sum of gradients (and hessians) and the count per bin, from which every candidate split's gain follows by cumulative sums. Only 15 candidates remain, and here the best one (134.3 kN) yields exactly the same partition of the rows as the exact search.

Why it is fast: binning happens once; per-node work is proportional to the number of bins, not rows; bins fit in one byte. The approximation is usually harmless, because thresholds between neighbouring values rarely matter for generalization. Transition: the order in which trees grow.

Libraries grow trees in different orders

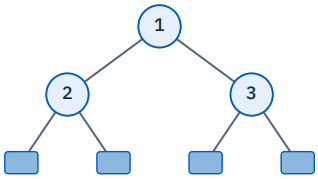
BOOSTING LIBRARIES AND STACKING

Libraries grow trees in different orders

Same three splits, different shapes: leaf-wise trees grow deep where the gain is.

Level-wise (depth-wise)

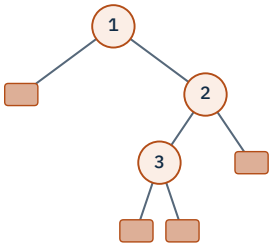
fills each level before going deeper



XGBoost default · limit: max_depth

Leaf-wise (best-first)

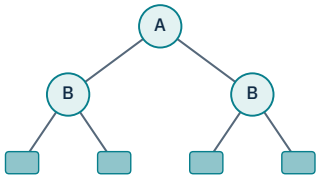
splits the leaf with the largest gain



LightGBM, sklearn HistGB · limit: leaves

Symmetric (oblivious)

one condition per level (A, then B)



CatBoost default · limit: depth

Schematic. Numbers give the order in which splits are made; A and B mark one condition shared across a level.

[3 min] Level-wise (depth-wise) growth, the default in XGBoost, splits every node of a level before going deeper; trees are balanced and the natural limit is max_depth.

Leaf-wise (best-first) growth, used by LightGBM and by scikit-learn's HistGradientBoosting, always splits the leaf with the largest gain, wherever it is. For the same number of leaves it reduces the loss more, but it can build deep, narrow branches that chase noise. Limit it by the number of leaves (num_leaves in LightGBM, max_leaf_nodes in scikit-learn, default 31 in both), plus a minimum number of rows per leaf.

Symmetric (oblivious) trees, the default in CatBoost, use one split condition for all nodes at the same depth. Each tree is a lookup table, fast at prediction time and strongly regularized.

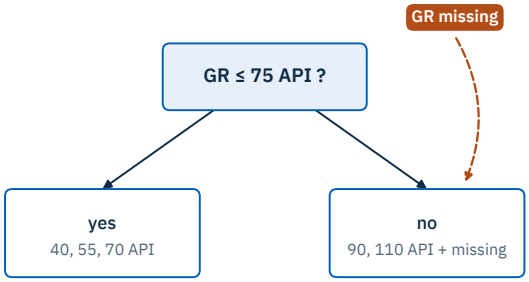
Takeaway: the same parameter name can mean different things (max_depth is secondary in a leaf-wise library). Check which growth policy your library uses before copying settings. Transition: missing values.

Missing values get a learned direction at each split

BOOSTING LIBRARIES AND STACKING

Missing values get a learned direction at each split

- No imputation needed: rows with a missing value go left or right.
- The direction is the one with the larger training gain.
- Categories split as groups, not as one-hot columns.



During training both directions are scored;
the one with the larger gain is stored.

No missing GR in training? sklearn sends them to the larger child.

◆ A gap is informative only if it is missing for the same reason in production.

Schematic of native missing-value handling in histogram gradient boosting.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 52

[3 min] At each split, the library evaluates the gain twice: once with all rows whose value is missing sent left, once sent right, and stores the better direction. At prediction time, a missing value follows the stored direction. If a feature had no missing values during training, scikit-learn sends missing values at prediction time to the child with more samples. XGBoost and LightGBM also learn a default direction.

Native categorical support (categorical_features in scikit-learn, pandas category dtype) lets a split send a group of categories left and the rest right, instead of expanding one-hot columns; categories are ordered by their gradient statistics to find good groupings.

The callout is the energy lesson. Missing values often carry information: a gauge fails in hot wells, or a log was not run in a section. The model will use that pattern. If the reason for missingness differs in production (a new telemetry system, a different logging programme), the learned direction becomes wrong. Document why values are missing. Transition: the code.

HistGradientBoosting handles gaps and categories natively

BOOSTING LIBRARIES AND STACKING

HistGradientBoosting handles gaps and categories natively

```
cols = ["WOB", "RPM", "FLOW", "BIT_HOURS", "FORMATION"]
names = ["Claystone", "Sandstone", "Limestone", "Dolomite"]
kinds = pd.CategoricalDtype(names)                    # fixed category list
X_tr = train[cols].astype({"FORMATION": kinds})
X_va = valid[cols].astype({"FORMATION": kinds})       # a validation well
print(X_tr.isna().mean().round(3).to_dict())

hgb = HistGradientBoostingRegressor(
    max_iter=1000, learning_rate=0.1, early_stopping=True,
    n_iter_no_change=20, random_state=0)
hgb.fit(X_tr, train["ROP"], X_val=X_va, y_val=valid["ROP"])   # sklearn 1.7
print("iterations used:", hgb.n_iter_)
```

OUTPUT

```
{'WOB': 0.0, 'RPM': 0.014, 'FLOW': 0.062, 'BIT_HOURS': 0.0, 'FORMATION': 0.0}
iterations used: 160
```

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 53

[4 min] Context: the synthetic drilling table with 1.4% of RPM and 6.2% of FLOW values removed at random (seed 5); train = wells D1–D4, valid = well D5; imports: pandas as pd and HistGradientBoostingRegressor.

Line by line: a fixed CategoricalDtype guarantees identical categories in training and validation data, so the category codes cannot silently differ; with the default categorical_features="from_dtype", scikit-learn treats that column as categorical. The missing values stay as NaN: no imputer. Early stopping uses the validation well passed as X_val and y_val, a feature added in scikit-learn 1.7; on older versions use validation_fraction (random rows) or a manual loop. The model stopped after 160 iterations.

Discuss: with one validation well, the stopping point depends on that well; with more time, rotate the validation well and average the chosen iteration counts. Ask: what would happen if production data spelled a formation "LIMESTONE"? It would not match the categories; validate and map inputs before prediction. Transition: comparing libraries.

Four libraries, one idea, different defaults

BOOSTING LIBRARIES AND STACKING

Four libraries, one idea, different defaults

Library	Default tree growth	Categorical inputs	Missing values
XGBoost	Level-wise (max_depth)	enable_categorical=True	Learned default direction
LightGBM	Leaf-wise (num_leaves)	categorical_feature	Learned direction
CatBoost	Symmetric (oblivious)	Ordered target statistics	Numeric NaN below minimum
scikit-learn HistGB	Leaf-wise (max_leaf_nodes)	categorical_features	Learned direction

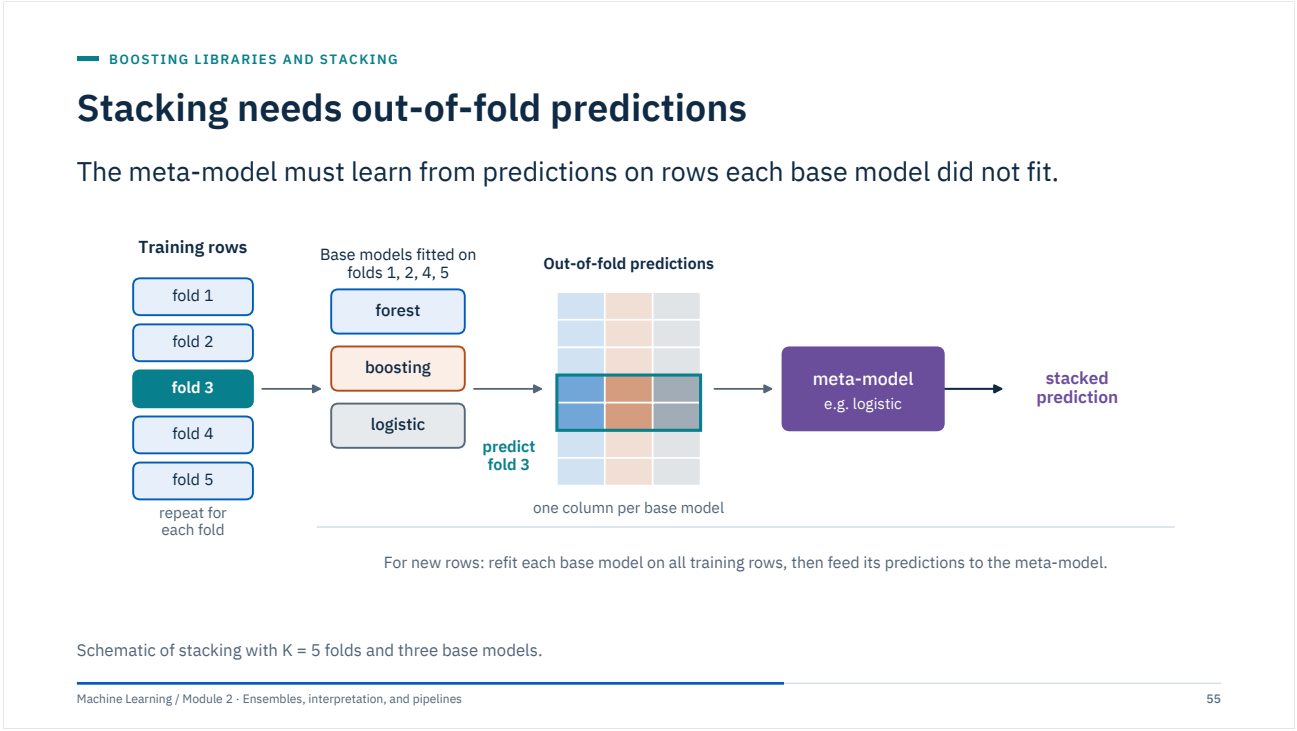
No universal ranking: compare on the same split, metric, hardware, and tuning budget.

[4 min] All four boost trees on binned (histogram) features, typically with second-order (Newton) leaf values; the differences are engineering and defaults.

XGBoost (Chen and Guestrin, 2016) grows level-wise by default and supports leaf-wise growth via grow_policy; native categorical support needs pandas categories and enable_categorical=True. LightGBM (Ke and colleagues, 2017) grows leaf-wise and adds sampling tricks for speed on large data. CatBoost (Prokhorenkova and colleagues, 2018) uses symmetric trees and encodes categories with ordered target statistics, computed so that a row's encoding never uses its own label; its default treats missing numeric values as smaller than every observed value. scikit-learn's HistGradientBoosting needs no extra installation and fits into pipelines and grouped cross-validation directly.

What to verify for any library: the objective and metric, the early-stopping data, the categorical schema at prediction time, and the version installed on the deployment machine. APIs change: consult the current official documentation (xgboost.readthedocs.io, lightgbm.readthedocs.io, catboost.ai/docs). Keep installations optional for the core lab. Transition: stacking.

Stacking needs out-of-fold predictions



[4 min] Stacking (Wolpert, 1992) trains a second-level model, the meta-model, to combine the predictions of several base models. The difficulty is training data for the meta-model. If base models predict the rows they were trained on, those predictions are unrealistically good (a deep forest predicts its own training rows almost perfectly), and the meta-model learns to trust the most overfitted model.

The fix, drawn here: split the training rows into K folds. For each fold, fit every base model on the other folds and predict the held-out fold. After K rounds every row has an out-of-fold prediction from every base model: one column per model. The meta-model, often a simple logistic or linear regression, is fitted on these columns. For new rows, each base model is refitted on all training rows and their predictions go through the meta-model.

Evaluate the complete stack on untouched test data. Retain simple averaging when the extra complexity lacks evidence. Standard stacking utilities may need customization for grouped or time-dependent data. Transition: the scikit-learn implementation.

StackingClassifier builds the out-of-fold features

BOOSTING LIBRARIES AND STACKING

StackingClassifier builds the out-of-fold features

```
members = [("forest", RandomForestClassifier(300, random_state=0)),
           ("hgb", HistGradientBoostingClassifier(random_state=0)),
           ("logit", make_pipeline(StandardScaler(), LogisticRegression()))]
stack = StackingClassifier(members, final_estimator=LogisticRegression(),
                           cv=5, stack_method="predict_proba")

for name, model in members + [("stack", stack)]:
    s = cross_val_score(model, X, y, groups=wells, cv=GroupKFold(4))
    print(f"{name:7s} {s.mean():.3f}")
```

OUTPUT

```
forest  0.776
hgb     0.820
logit   0.857
stack   0.852
```

⚠ The internal cv=5 splits rows, not wells: its out-of-fold columns still leak within wells.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 56

[3 min] Context: the eight-well synthetic facies data; the outer loop evaluates every candidate with the same grouped folds (two whole wells held out per fold).

Result: the stack scores 0.852, slightly below the logistic regression alone (0.857) and well above the forest (0.776). Stacking did not earn its complexity here. One reason is visible in the warning: the stack's internal cv=5 splits rows, so the out-of-fold predictions of the forest still benefit from neighbouring rows of the same well and look better than they will on a new well; the meta-model then over-trusts the forest.

For grouped or time-ordered data, build the out-of-fold matrix yourself: cross_val_predict(model, X, y, groups=wells, cv=GroupKFold(...), method="predict_proba") for each base model, stack the columns, fit the meta-model, and evaluate the whole procedure inside an outer grouped loop. Keep simple averaging, or the single best model, when the evidence does not favour the stack. Transition: an exercise on choosing.

Choosing forest or boosting

PRACTICE

5 min Groups of 3

Choosing forest or boosting

A tabular model will run at a remote wellsite. You have limited tuning time.

1. Which baseline would you try first, and why?
2. What comparison would make you change your choice?
3. What must fit within the deployment budget?

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 57

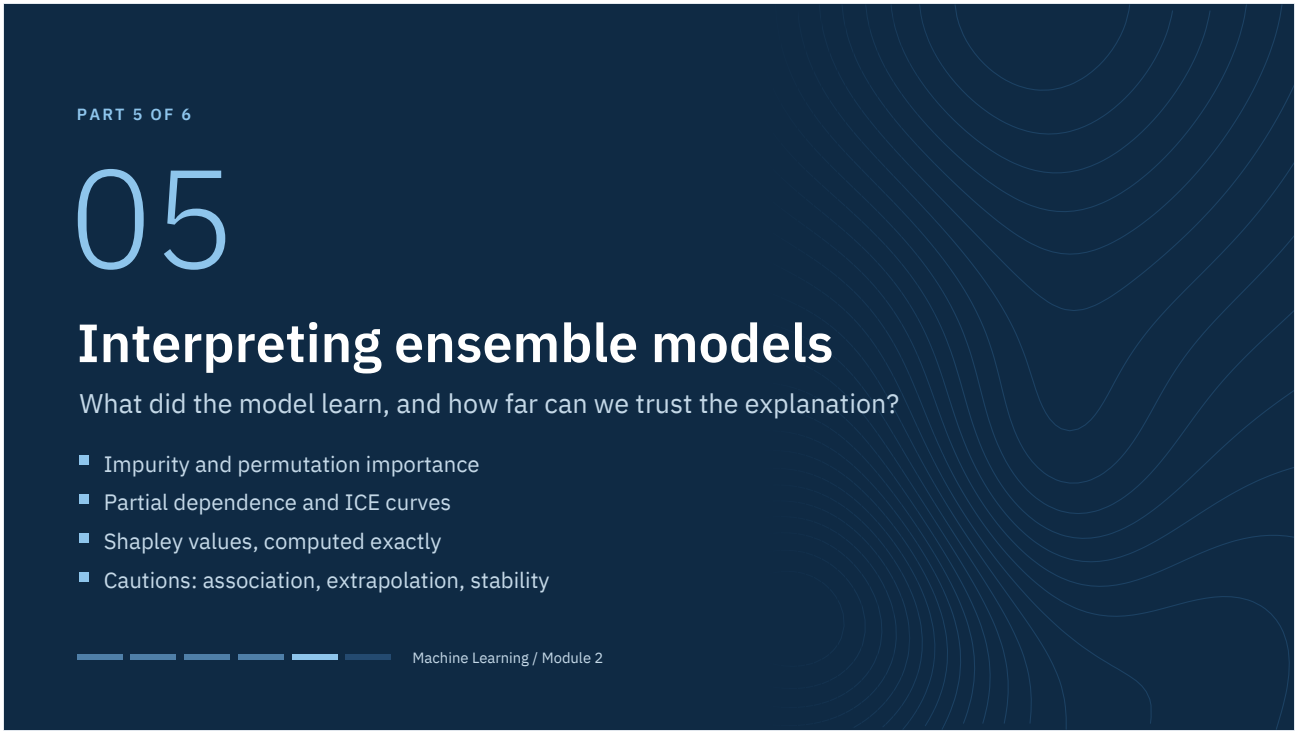
[5 min] There is no single correct algorithm; grade the reasoning, not the algorithm name.

A random forest is a defensible first nonlinear baseline: few sensitive settings, robust defaults, a free OOB check on independent rows, and no stopping decision. Boosting is also defensible if the team already has a reliable workflow, especially HistGradientBoosting, which handles missing sensor values and categories natively and trains quickly.

A comparison that should change the choice: both candidates evaluated on the same grouped folds (by well or by time period), with a paired difference that exceeds the fold-to-fold variation and matters operationally (for example, fewer missed trips at the same false-alarm rate).

Deployment budget: model size and memory on the edge device (a 500-tree forest of deep trees can be hundreds of megabytes; boosted ensembles of shallow trees are usually much smaller), prediction latency, behaviour with missing inputs, retraining frequency, and who maintains the code. Look for answers that mention maintainability and monitoring, not only accuracy. Transition: Part 5, interpretation.

Interpreting ensemble models



Part 5 [45 min]. Ensembles are accurate but opaque: nobody can read 300 trees. Stakeholders still ask which inputs matter, how the prediction responds to a setting such as weight on bit, and why one particular prediction is low. This part gives three families of answers: global importance (which features the model relies on), effect curves (how predictions change with a feature), and local attributions (how one prediction is split among features).

Every method answers a narrow question about a fitted model on particular data. None of them identifies physical cause and effect, and each has a failure mode that we will demonstrate on synthetic data where the truth is known.

Ask before starting: what would a drilling engineer do with a statement like "WOB is the most important feature"? Keep the answers; we will see whether the statement survives correlated inputs. Transition: importance depends on the question.

Feature importance depends on the question

INTERPRETING ENSEMBLE MODELS

Feature importance depends on the question

Method	Measures	Main limitation
Split impurity importance	Training impurity reduction credited to a feature	Can favour high-cardinality predictors
Permutation importance	Evaluation score drop after shuffling a feature	Correlated substitutes can hide reliance
Domain sensitivity review	Plausibility of model response to input changes	Requires careful constraints

[3 min] Impurity importance, also called mean decrease in impurity (MDI), is what scikit-learn stores in feature_importances_: for every split, the decrease in impurity weighted by the rows reaching that node, summed per feature and averaged over trees. It is computed on training data and is free, but it credits any feature the trees happened to split on, including splits that only fit noise.

Permutation importance shuffles one column in evaluation data and measures how much the score drops. It describes reliance of a particular fitted model on particular evaluation data. Compute permutation importance after confirming the model has useful performance; importance of a useless model is meaningless.

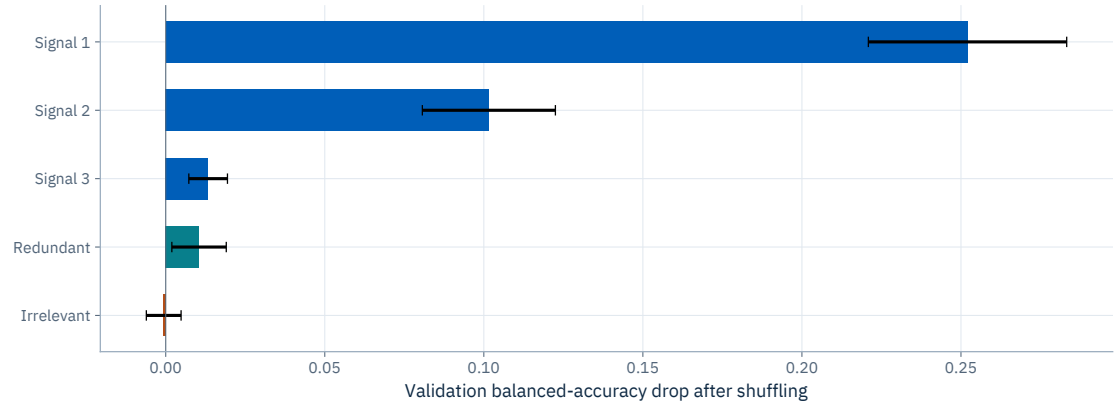
Domain sensitivity review asks whether the response to controlled input changes is physically plausible, for example ROP should not fall when only the bit is sharper. Neither kind of importance identifies causal effects. Use grouped permutations for correlated feature sets when appropriate. Transition: permutation importance on a known example.

Importance can expose an irrelevant feature

INTERPRETING ENSEMBLE MODELS

Importance can expose an irrelevant feature

Shuffle one column in the validation rows; the score drop measures the model's reliance on it.



Computed synthetic validation experiment with a deliberately irrelevant feature. Error bars show variation over repeated shuffles.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 60

[2 min] Read the metric axis carefully: a larger drop implies greater model reliance under this evaluation. The three signal features have clear drops; the redundant feature (a linear combination of signals) has a small drop because the model can recover its information from the others; the irrelevant feature sits at zero, within the error bar.

Small negative values can arise from sampling noise or a harmful feature: shuffling it made the score slightly better by chance. Repeat the shuffles (n_repeats) and read the spread before interpreting any small value.

Do not treat this figure as an energy dataset result. Ask: why must this be computed on validation rather than training rows? On training rows an overfitted model relies on noise features too, so they would look important. Transition: what goes wrong with impurity importance.

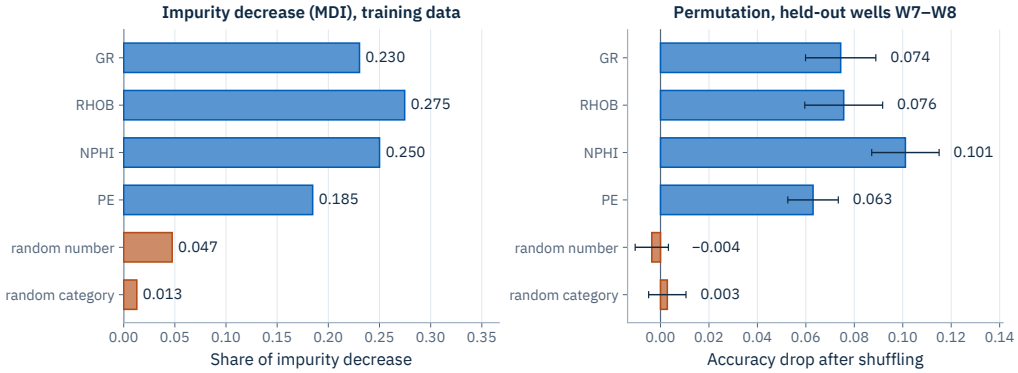
Reference: https://scikit-learn.org/stable/modules/permutation_importance.html

Impurity importance rewards high-cardinality noise

INTERPRETING ENSEMBLE MODELS

Impurity importance rewards high-cardinality noise

Random numbers get a quarter of PE's impurity credit, yet shuffling them changes nothing on new wells.



Synthetic facies logs plus two pure-noise columns (seed 0); forest of 300 unpruned trees trained on W1–W6; permutation on W7–W8, 20 repeats.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 61

[4 min] We added two columns of pure noise to the facies logs: a random number (continuous, every value unique) and a random category with three levels. Left: impurity importance gives the random number 0.047, a quarter of PE's 0.185, and more than three times the random category's 0.013. Right: permutation importance on held-out wells gives the random number −0.004 and the category 0.003, both indistinguishable from zero, while the four logs score between 0.06 and 0.10.

Why MDI is biased: a continuous feature offers hundreds of candidate thresholds, so deep trees can always find some split on it that reduces training impurity a little, especially in small nodes near the leaves. The credit is real on training data and useless elsewhere. The bias toward high-cardinality features is documented by Strobl and colleagues (2007) and in the scikit-learn user guide.

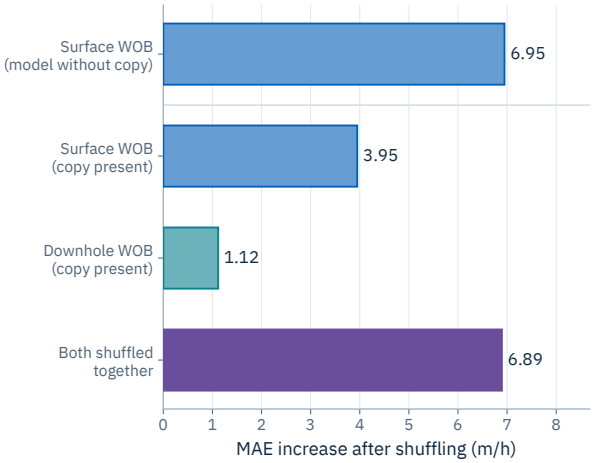
Practice: report permutation importance on held-out data (grouped where needed) and treat feature_importances_ only as a quick training diagnostic. Transition: correlated features.

Correlated features share and hide importance

INTERPRETING ENSEMBLE MODELS

Correlated features share and hide importance

- Downhole WOB is a near copy of surface WOB (r = 0.997).
- With both present, shuffling one leaves the other: 3.95 + 1.12.
- Shuffling them together recovers the joint effect: 6.89.



◆ Group correlated inputs before ranking them.

Synthetic drilling data; forest trained on D1–D4, permutation on D5–D6, 30 repeats.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 62

[4 min] Top bar: a forest trained without the copy relies on surface weight on bit; shuffling it raises the mean absolute error on wells D5–D6 by 6.95 m/h. Then we add a downhole WOB column, 0.92 × surface WOB plus noise (correlation 0.997), as a downhole sub would measure. With both present, shuffling surface WOB costs only 3.95 m/h and shuffling downhole WOB 1.12 m/h, because the trees can use whichever copy was not shuffled. Neither number reflects how much the model depends on weight on bit. Shuffling both columns with the same permutation restores the joint importance: 6.89 m/h.

Remedies: permute correlated groups together; or cluster features by correlation (for example on Spearman rank correlation) and keep one per cluster before ranking; always report which features were grouped.

Return to the question from the section slide: "WOB is the most important feature" can be false, true, or split in two depending on which measurement columns the model sees. The ranking describes the model's inputs, not the physics. Transition: the code for importance and dependence.

Importance and dependence plots take a few lines

INTERPRETING ENSEMBLE MODELS

Importance and dependence plots take a few lines

```
from sklearn.inspection import permutation_importance
from sklearn.inspection import PartialDependenceDisplay

r = permutation_importance(model, X_val, y_val, n_repeats=20, random_state=0,
                           scoring="neg_mean_absolute_error")
for name, m, s in zip(X_val.columns, r.importances_mean, r.importances_std):
    print(f"{name:10s} {m:5.2f} ± {s:.2f}")

PartialDependenceDisplay.from_estimator(
    model, X_val, features=["WOB", "RPM"], kind="both",
    subsample=70, random_state=0)
```

OUTPUT

```
WOB         6.87 ± 0.33
RPM         1.83 ± 0.13
FLOW        0.48 ± 0.04
BIT_HOURS   0.13 ± 0.03
FORMATION   8.99 ± 0.39
```

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 63

[2 min] Context: model is a HistGradientBoostingRegressor (FORMATION categorical) trained on wells D1–D4; X_val and y_val are wells D5–D6. With scoring="neg_mean_absolute_error", importances are increases in MAE, in m/h, which people can interpret directly.

Read the output: formation and weight on bit dominate, rotary speed matters less, flow and bit hours very little. The ± values are standard deviations over 20 shuffles; they show the shuffling noise, not the uncertainty from choosing these two wells. Rerunning on other validation wells is the honest way to see how stable the ranking is.

The second call draws the partial-dependence curve (average) and 70 ICE curves (individual rows) for WOB and RPM, a plot like the one on the next slide. subsample limits the number of ICE lines drawn; random_state makes the choice reproducible.

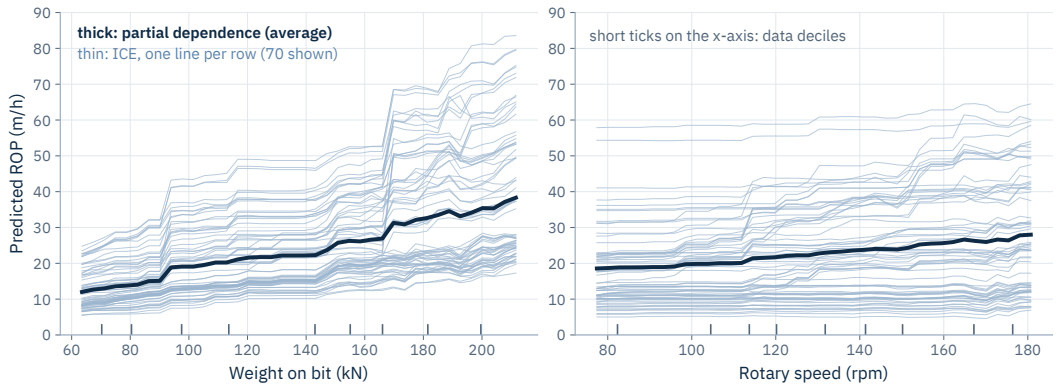
Common error: passing the training rows to permutation_importance, which measures reliance including overfitting. Transition: reading the curves.

Partial dependence averages the model's response

INTERPRETING ENSEMBLE MODELS

Partial dependence averages the model's response

PD: set WOB to each grid value for every row, predict, and average. ICE: keep one line per row.



HistGradientBoostingRegressor trained on synthetic wells D1–D5; sklearn.inspection.partial_dependence over 300 sampled rows, 40 grid points between the 5th and 95th percentiles.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 64

[4 min] How a partial-dependence (PD) curve is made (Friedman, 2001): take the evaluation rows, set every row's WOB to the same grid value, predict, and average; repeat for each grid value. The thick line is that average. Individual conditional expectation (ICE) curves (Goldstein and colleagues, 2015) keep each row's line separately instead of averaging.

Left: predicted ROP rises with weight on bit for almost every row, with steps where the trees split. Right: the average rise with rotary speed is modest, but the ICE lines fan out: some rows rise steeply, many stay flat. The average hides that the model's response depends on something else.

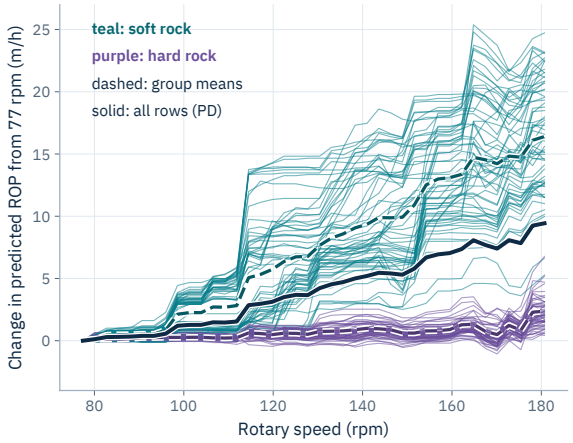
Reading tips: the short ticks on the x-axis are data deciles; curves beyond the data range are extrapolation. The staircase shape comes from the trees, not from physics. These are properties of the fitted model on synthetic data, not a drilling response curve. Transition: splitting the ICE lines by formation.

ICE curves reveal what the average hides

INTERPRETING ENSEMBLE MODELS

ICE curves reveal what the average hides

- Centre each curve at the lowest rpm to compare slopes.
- Soft rock: predicted ROP rises with rpm; hard rock: almost flat.
- The average curve describes neither group.



◆ Diverging ICE curves signal an interaction: plot them by group.

Centred ICE curves from the synthetic-drilling boosting model; 120 of 300 rows shown, coloured by formation.

[3 min] We centred every ICE curve at its value at 77 rpm, so the plot shows how much each row's prediction changes as rotary speed increases. Colour shows formation: teal for claystone and sandstone, purple for limestone and dolomite.

The pattern is striking: across the rpm range the model's predicted ROP rises by 16 m/h on average in soft rock (up to 25 m/h for some rows), but only by 2.4 m/h in hard rock. The partial-dependence curve (black) rises by 9.4 m/h, in between, and describes neither group. This is an interaction between rotary speed and formation, and the generator that produced these synthetic data does contain it, so here the model recovered a real (synthetic) effect.

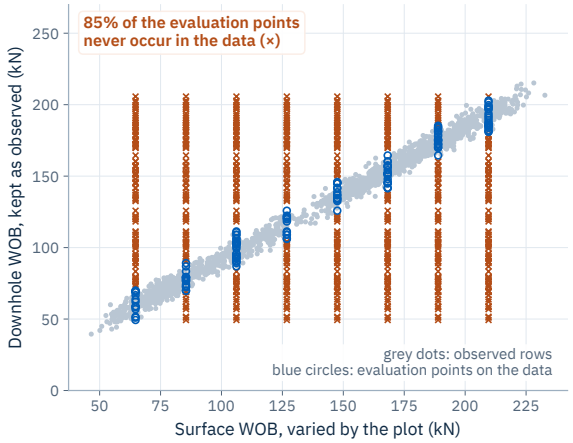
On field data you would not know the truth. Diverging ICE curves tell you that an interaction exists in the model; they do not tell you that it exists in the rock. Next step: check that both groups are well represented in training data, and ask a drilling engineer whether the pattern is plausible. Transition: when PD itself misleads.

Partial dependence can query impossible inputs

INTERPRETING ENSEMBLE MODELS

Partial dependence can query impossible inputs

- PD pairs each grid value with every observed value of the other inputs.
- With correlated inputs most pairs never occur: 85% here.
- The model's answers there are extrapolation, not evidence.



◆ Restrict to observed combinations, or use accumulated local effects.

Synthetic drilling data with a downhole WOB column; 8 grid values × 120 observed downhole values.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 66

[3 min] Suppose a model uses both surface and downhole WOB. The PD curve for surface WOB sets surface WOB to, say, 200 kN for every row while keeping each row's downhole value, which may be 60 kN. The scatter shows the problem: the observed rows (grey) lie on a narrow diagonal band, but the evaluation points form vertical columns across the whole range. 85% of them (crosses) lie farther from any observed row than 99% of real rows are from their nearest neighbour: combinations no tool would ever record.

The model's predictions at those points are extrapolation; trees simply return whatever leaf the point falls into. Averaging them into a PD curve mixes evidence with fiction.

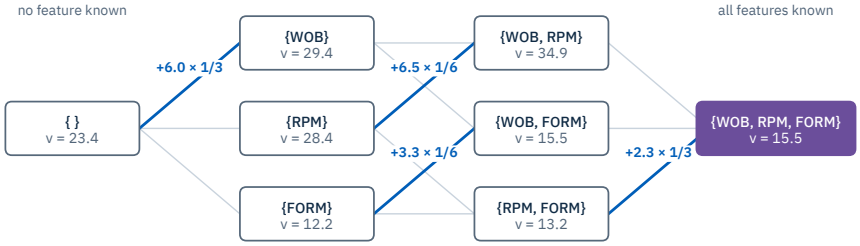
Remedies: check pairwise correlations before plotting; plot PD only for features that are nearly independent of the rest, or group correlated features; use accumulated local effects (ALE; Apley and Zhu, 2020), which average local changes only among rows with similar values; or compare with conditional plots drawn from the data. Transition: explaining one prediction.

Shapley values average a feature's marginal steps

INTERPRETING ENSEMBLE MODELS

Shapley values average a feature's marginal steps

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\ (p - |S| - 1)!}{p!} \left[v(S \cup \{j\}) - v(S) \right]$$



φ(WOB) = weighted sum of the four blue steps = +4.43 m/h · v(S): mean prediction with the features in S fixed to this row

Exact computation for one row of well D6; 3-feature boosting model; v(S) in m/h over 200 background rows.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 67

[5 min] A Shapley value (Shapley, 1953; applied to models by Lundberg and Lee, 2017) splits one prediction among the features. Define v(S) as the model's average prediction when the features in S are fixed to this row's values and the others are taken from background rows. Here: v({}) = 23.4 m/h (nothing known), v({WOB, RPM, FORM}) = 15.5 m/h (this row's prediction).

Every path from the empty set to the full set adds features one at a time. The blue edges are the steps that add WOB: +6.0 from {} (weight 1/3), +6.5 from {RPM} (1/6), +3.3 from {FORM} (1/6), and +2.3 from {RPM, FORM} (1/3). The weights count the orderings in which WOB joins after exactly that set. φ(WOB) = 6.047/3 + 6.538/6 + 3.295/6 + 2.330/3 = 2.016 + 1.090 + 0.549 + 0.776 = 4.43 m/h.

Symbols: F is the full feature set, p = 3 features, S a subset not containing j. The step sizes differ because of interactions: adding WOB helps less once dolomite is known. Transition: the complete explanation of this row.

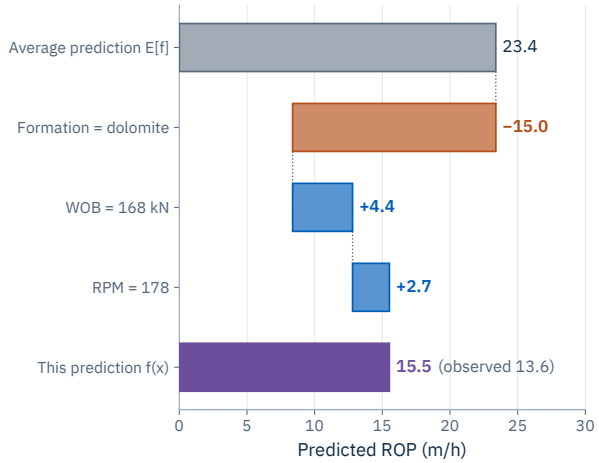
Shapley values add up to the prediction

INTERPRETING ENSEMBLE MODELS

Shapley values add up to the prediction

$$f(x) = \mathrm{E}[f(X)] + \sum_{j=1}^{p} \phi_j$$

- Base: the average prediction over the background rows, 23.4 m/h.
- Dolomite −15.0, WOB +4.4, RPM +2.7: this row predicts 15.5 m/h.
- Features outside S are filled from background rows (interventional).



Exact interventional Shapley values for one row of synthetic well D6.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 68

[3 min] The waterfall reads top to bottom: start at the background average, 23.4 m/h; being in dolomite lowers the prediction by 15.0; weight on bit of 168 kN raises it by 4.4; 178 rpm raises it by 2.7; the result is exactly the model's prediction, 15.5 m/h (observed ROP at that depth: 13.6). This additivity (efficiency) is one of the properties that define Shapley values; the others are symmetry, a zero value for features that never change the output, and linearity across models.

Interpretation traps. The RPM contribution is positive even though the generator gives rpm little effect in hard rock: the value averages over coalitions in which the formation is unknown, where high rpm raises the average prediction. The values depend on the background data; change the background (for example, only hard-rock rows) and the base and contributions change.

Cost: exact computation needs 2^p coalitions, each averaged over the background; here 8 × 200 predictions. For trees, TreeSHAP computes exact values efficiently; for other models, sampling approximations are used (the shap package). Transition: an exercise.

Shapley values by hand

PRACTICE

5 min Pairs

Shapley values by hand

A two-feature ROP model gives average predictions (m/h): no feature known 20; WOB known 26; RPM known 22; both known 30.

1. Compute the Shapley value of WOB and of RPM.
2. Check that they add up to the prediction minus the base.
3. Why is WOB's step larger once RPM is known?

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 69

[5 min] With two features there are two orderings, each with weight 1/2.

(1) WOB first: WOB adds 26 − 20 = 6, then RPM adds 30 − 26 = 4. RPM first: RPM adds 22 − 20 = 2, then WOB adds 30 − 22 = 8. φ(WOB) = ½(6 + 8) = 7 m/h. φ(RPM) = ½(2 + 4) = 3 m/h.

(2) 7 + 3 = 10 = 30 − 20. The base (20) plus the contributions gives the prediction (30).

(3) Because of an interaction: in this model WOB is worth more when RPM is also high (8 instead of 6). Shapley values share the interaction term, 8 − 6 = 2, equally between the two features: each gets half of it. That is a convention, not a physical measurement.

Common errors: reporting a single ordering (6 and 4, or 8 and 2), which depends on an arbitrary choice; forgetting the weights; treating the base as zero. Extension: with three features there are 3! = 6 orderings and the weights are 1/3, 1/6, 1/6, 1/3 as on the lattice slide. Transition: choosing an explanation method.

Choose the explanation that answers your question

INTERPRETING ENSEMBLE MODELS

Choose the explanation that answers your question

Method	Question it answers	Scope	Main pitfall
Impurity (MDI)	Which splits cut training impurity?	Global, training	Rewards high-cardinality noise
Permutation	How much does the score rely on a feature?	Global, held-out	Correlated inputs share credit
PD and ICE	How do predictions change with one input?	Average and per row	Extrapolates with correlated inputs
Shapley values	How is one prediction split among inputs?	One row	Depends on the background data

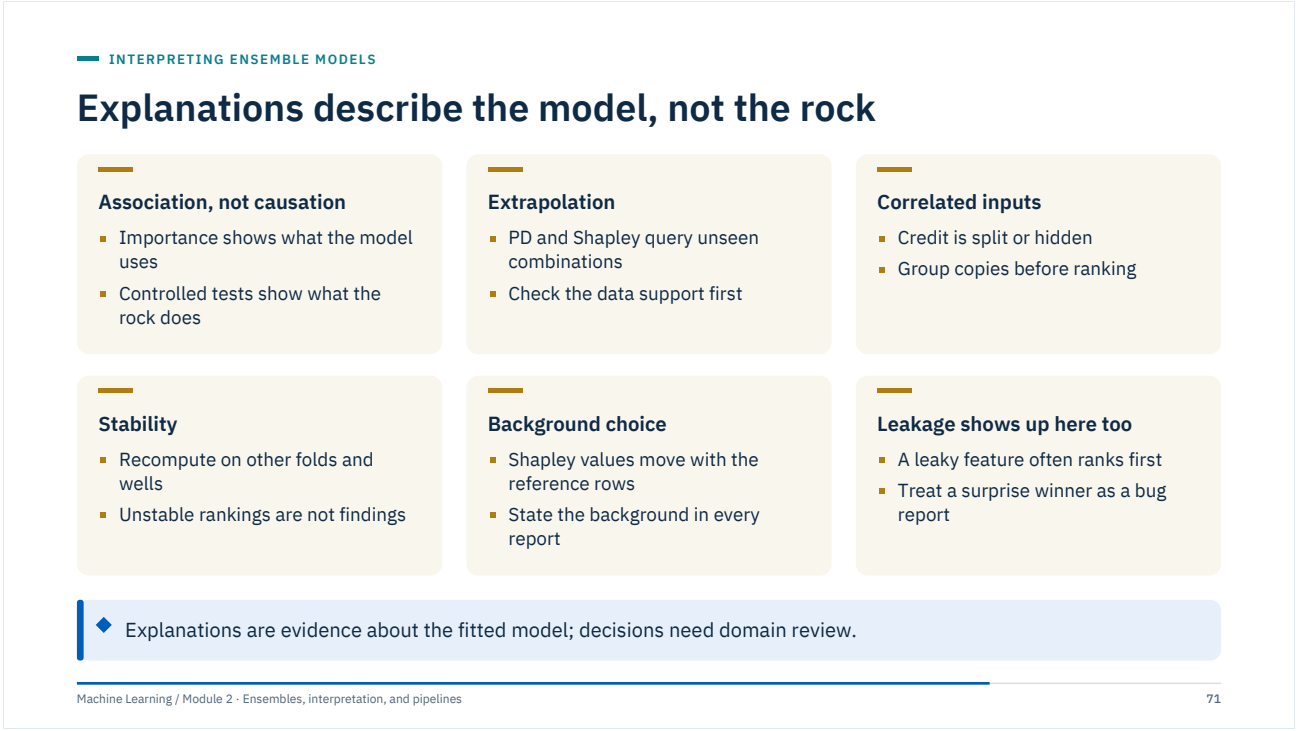
Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 70

[3 min] Use this table as a decision aid. Start from the stakeholder's question, not from the method you know best.

"Which measurements should we keep paying for?" is a reliance question: permutation importance on held-out, deployment-like data, with correlated sensors grouped. "What happens to predicted ROP if we raise WOB?" is an effect question: PD with ICE, restricted to realistic input combinations, and remembering that the answer describes the model, not a controlled field test. "Why is this well's forecast so low?" is a local question: Shapley values for that row, with a documented background dataset. "Is anything weird in training?" is a diagnostic question where impurity importance is a cheap first look.

Stability check for every method: recompute on different folds or wells. If rankings or curve shapes change, report the variation or do not report a ranking at all. Transition: the cautions in one slide.

Explanations describe the model, not the rock

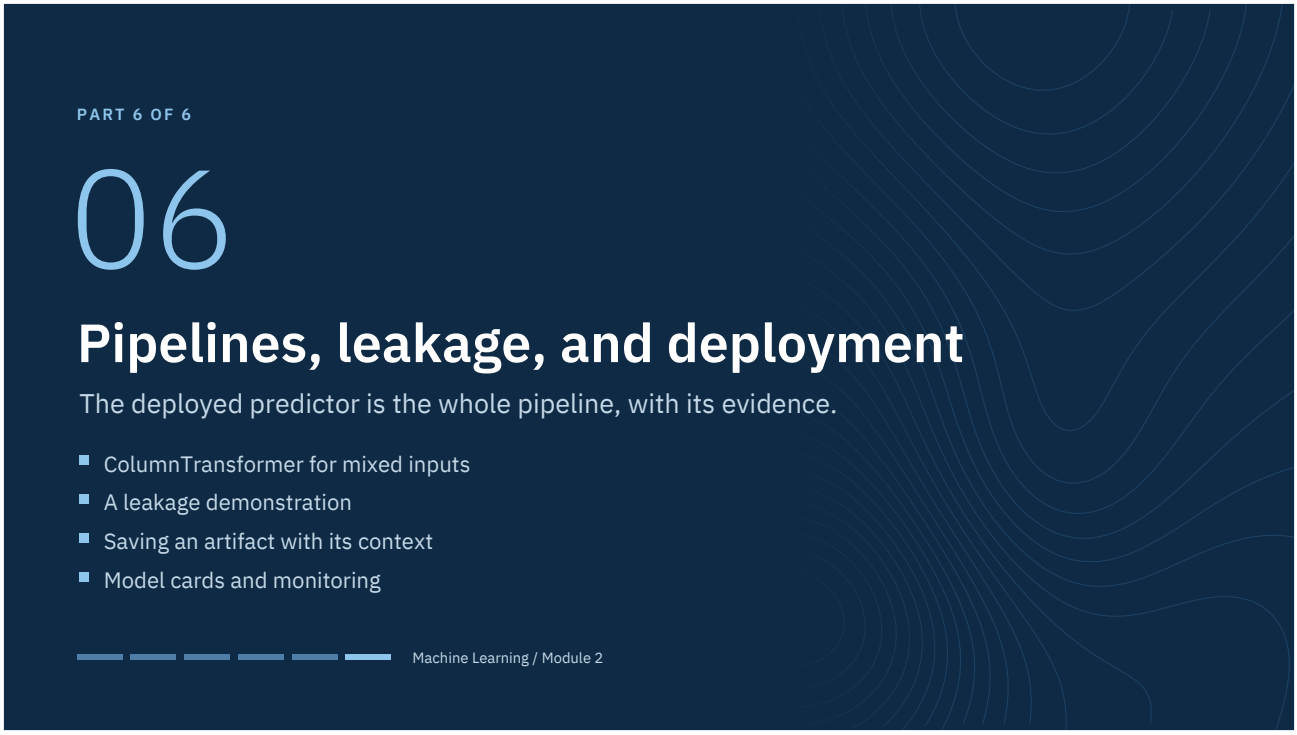


[3 min] Summarize the part with the six cautions. Association, not causation: a model can rely on a feature because it correlates with the true cause, for example bit hours proxying depth and formation; changing that feature in the field need not change the outcome. Extrapolation: PD and Shapley values evaluate the model where it has no data. Correlated inputs: credit moves between copies. Stability: explanations inherit the variance of the model; recompute them across folds. Background choice: Shapley values are relative to a reference population, which must be stated.

The last card is a useful habit: when an unexpected feature tops the importance ranking (a record ID, a timestamp, a "days since failure" column), investigate for leakage before celebrating. Many leaks in Module 1's examples would have been caught this way.

Ask: which caution matters most for the facies model we saw earlier? (Stability across wells, and correlated logs such as RHOB and NPHI.) Transition: Part 6, pipelines.

Pipelines, leakage, and deployment



Part 6 [35 min]. Module 1 introduced pipelines for numeric preprocessing. Real energy tables mix numeric sensors, categorical labels (formation, equipment type), and metadata (well IDs, timestamps). This part builds a ColumnTransformer for such a table, demonstrates how selection outside cross-validation creates a fake signal, tunes the complete pipeline, and saves it in a form that can be audited and reloaded.

Ask learners to name what would be missing from a saved model object if scaling or encoding had occurred in a notebook cell outside the model pipeline. Expected answers: the fitted means and scales, the category list, the column order, and the imputation values. Without them, a reloaded model silently receives differently prepared inputs.

The part ends with the documentation that should travel with a model (a model card) and the monitoring that begins after release, with detailed drift methods left to Module 5. Transition: mixed data.

Mixed data needs separate transformations

PIPELINES, LEAKAGE, AND DEPLOYMENT

Mixed data needs separate transformations

Input family	Example	Training transformation
Numeric	Pressure, flow, gamma ray	Imputation and optional scaling
Categorical	Formation or equipment type	Imputation and category encoding
Metadata	Well ID, timestamp, record ID	Split, join, or ordering key

New categories and missing sensors should have defined behaviour.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 73

[2 min] Do not automatically make all metadata predictors. A well ID can encode memorization. A timestamp can encode trends but must match forecasting assumptions. Define units and available features before writing the pipeline.

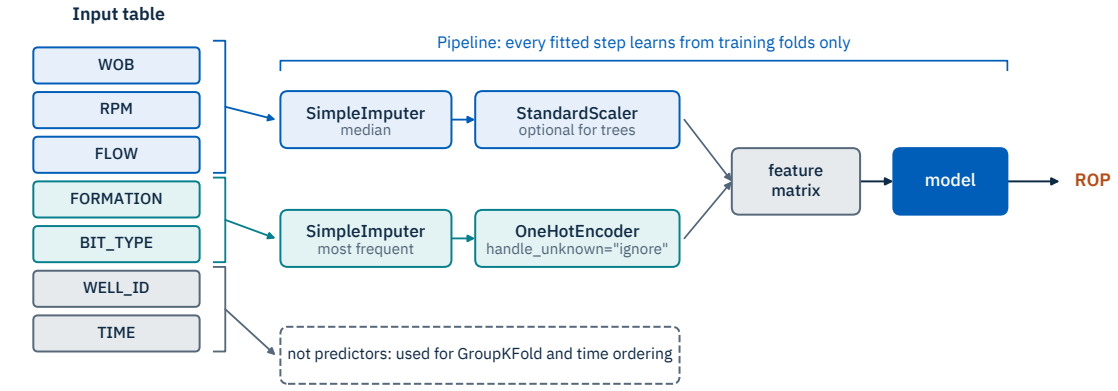
Walk the rows. Numeric inputs need a rule for gaps (median imputation, or native NaN handling in histogram boosting) and scaling only for models that are sensitive to scale (linear models, KNN, neural networks; not trees). Categorical inputs need a rule for missing and for new categories: a formation name never seen in training must not crash the service or be silently mapped to the wrong code. Metadata columns drive the validation design (GroupKFold on well ID, chronological splits on time) and the joins, but are usually not predictors.

Ask: what should happen when a new bit type appears in production? A defined behaviour: an "unknown" code, a warning, and a count on the monitoring dashboard. Transition: a ColumnTransformer drawn out.

A ColumnTransformer routes each column family

PIPELINES, LEAKAGE, AND DEPLOYMENT

A ColumnTransformer routes each column family



Schematic of a ColumnTransformer inside a Pipeline for the synthetic drilling table; column names are illustrative.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 74

[3 min] Follow the colours. Blue numeric columns (WOB, RPM, FLOW) pass through a median imputer and, optionally, a scaler; for tree models the scaler has no effect and can be omitted. Teal categorical columns (FORMATION, BIT_TYPE) pass through a most-frequent imputer and a one-hot encoder that ignores unknown categories at prediction time. Grey metadata columns (WELL_ID, TIME) are not predictors: they are kept outside the feature matrix and used for GroupKFold and time ordering.

The outputs are concatenated into one feature matrix and passed to the model. The blue bracket is the important part: every step inside it is fitted on training folds only. During cross-validation the imputation medians, the category lists, and the model are all re-learned inside each fold, so validation rows never influence preprocessing.

Note: ColumnTransformer's default remainder="drop" silently drops every column not listed, which is exactly what we want for metadata, but also means a new predictor must be added to a column list explicitly. Transition: the code.

A ColumnTransformer preserves the schema

PIPELINES, LEAKAGE, AND DEPLOYMENT

A ColumnTransformer preserves the schema

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline

numeric = make_pipeline(SimpleImputer(strategy="median"))
categorical = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore")
)
prep = ColumnTransformer([
    ("num", numeric, numeric_columns),
    ("cat", categorical, categorical_columns)
])
```

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 75

[2 min] The column lists are explicit lists of predictor names from the training schema, for example numeric_columns = ["WOB", "RPM", "FLOW", "BIT_HOURS"] and categorical_columns = ["FORMATION"]. Selecting columns by name, not by position, protects against reordered inputs.

OneHotEncoder produces sparse output by default; select an estimator and pipeline that support the resulting representation. For estimators requiring dense arrays, set sparse_output=False after considering memory cost. handle_unknown="ignore" encodes an unseen category as all zeros, which keeps the service running; count such rows in monitoring, because the model has no information about them.

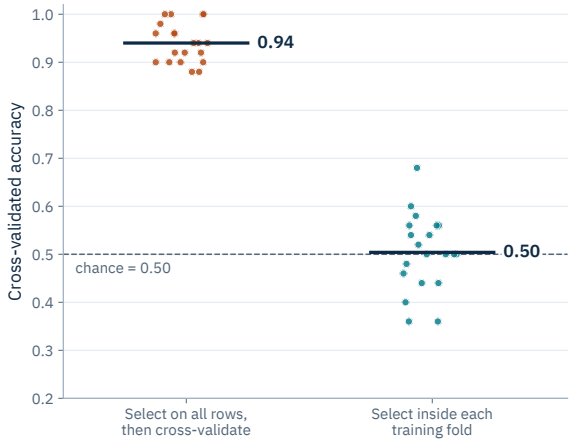
Columns not named in either list are dropped (remainder="drop"), so WELL_ID and TIME do not become predictors by accident. After fitting, prep.get_feature_names_out() lists the output columns, useful for importance tables and model cards. Transition: a leakage demonstration.

Selecting features on all rows fakes a signal

PIPELINES, LEAKAGE, AND DEPLOYMENT

Selecting features on all rows fakes a signal

- 50 rows, 2,000 noise features, random labels: nothing to learn.
- Select the 20 "best" features on all rows, then cross-validate: 0.94.
- Select inside each training fold: 0.50, which is chance.



◆ Any step that looks at labels must sit inside the pipeline.

20 synthetic datasets (seed 0), SelectKBest(f_classif, k = 20) and logistic regression; 5-fold stratified CV.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 76

[4 min] This experiment follows the "wrong way to do cross-validation" example in Hastie, Tibshirani and Friedman (section 7.10.2). Each dataset has 50 rows, 2,000 features of pure Gaussian noise, and random balanced labels: no model can beat 50% on new data.

Orange: select the 20 features most associated with the labels using all 50 rows, then cross-validate a logistic regression on those 20. Mean accuracy 0.94 across 20 repetitions. With 2,000 candidates, some features correlate with the labels by chance in these 50 rows, including the rows that later serve as validation folds; the selection step has already seen the answers.

Teal: the selector is inside the pipeline, refitted on each training fold. Mean accuracy 0.50, exactly chance, with the spread (0.36 to 0.68) expected from 10-row validation folds.

Energy relevance: selecting "the most predictive sensors" from hundreds of tags, or choosing lag features, on the full dataset before validation creates the same illusion. Transition: the code.

Feature selection belongs inside the pipeline

PIPELINES, LEAKAGE, AND DEPLOYMENT

Feature selection belongs inside the pipeline

```
rng = np.random.default_rng(0)
X = rng.normal(size=(50, 2000))               # 2,000 pure-noise features
y = rng.permutation(np.repeat([0, 1], 25))    # random labels
cv = StratifiedKFold(5, shuffle=True, random_state=0)

# Wrong: the selector sees every row, including each validation fold
X_sel = SelectKBest(f_classif, k=20).fit_transform(X, y)
wrong = cross_val_score(LogisticRegression(), X_sel, y, cv=cv)

# Right: selection is refitted inside every training fold
pipe = make_pipeline(SelectKBest(f_classif, k=20), LogisticRegression())
right = cross_val_score(pipe, X, y, cv=cv)
print(f"wrong {wrong.mean():.2f}   right {right.mean():.2f}")
```

OUTPUT

```
wrong 0.90   right 0.36
```

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 77

[3 min] This is one of the datasets behind the previous figure, written out in full so learners can run it. Imports: numpy as np; SelectKBest and f_classif from sklearn.feature_selection; LogisticRegression; StratifiedKFold and cross_val_score; make_pipeline.

The wrong version fits the selector on all 50 rows, so the chosen 20 columns already encode the labels of every validation fold: 0.90. The right version refits the selector on 40 training rows in each fold: 0.36. That value is below chance by luck: each validation fold has only 10 rows, and across the 20 datasets of the figure the right version averages 0.50.

Generalize the rule: any step that uses labels (feature selection, target encoding, resampling for imbalance, threshold tuning) must be inside the cross-validation loop, and any step that uses data statistics (scaling, imputation, PCA) should be too. The pipeline makes this automatic. Transition: tuning the whole pipeline.

Tuning operates on the complete pipeline

PIPELINES, LEAKAGE, AND DEPLOYMENT

Tuning operates on the complete pipeline

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline

model = Pipeline([
    ("prep", prep),
    ("forest", RandomForestClassifier(
        n_estimators=200, random_state=42, n_jobs=-1))
])
param_grid = {
    "forest__min_samples_leaf": [1, 5, 15],
    "forest__max_features": ["sqrt", 0.7]
}
```

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 78

[3 min] Use the grouped search pattern from Module 1, adapting parameter names: GridSearchCV(model, param_grid, cv=GroupKFold(n_splits=5), scoring=...) and search.fit(X_train, y_train, groups=wells_train). Parameter names combine the step name and the parameter with a double underscore: forest__min_samples_leaf. Preprocessing parameters can be tuned the same way, for example prep__num__simpleimputer__strategy.

Because the whole pipeline is refitted in every fold, imputation and encoding are learned from training folds only, and the best estimator (search.best_estimator_) is a complete, fitted predictor, ready to save.

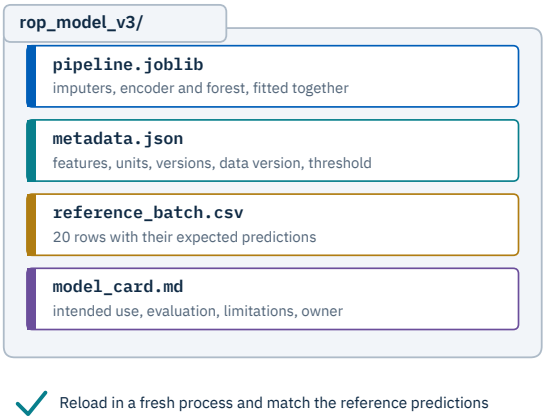
For nested parallel searches, avoid assigning every CPU core at both levels. Start with a search budget the class can finish on available machines. Search strategies (random, successive halving, Bayesian) and nested cross-validation for an unbiased estimate of the tuned model are Module 5 topics. Transition: what to save.

The saved artifact needs context

PIPELINES, LEAKAGE, AND DEPLOYMENT

The saved artifact needs context

- Save fitted preprocessing and estimator together.
- Record feature names, units, versions, data version, and threshold.
- Reload in a fresh process and compare predictions on a reference batch.
- Load serialized Python objects only from trusted sources.



Schematic of a versioned model artifact; file names are illustrative.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 79

[3 min] Model persistence mechanisms such as joblib and pickle can execute code during loading. This is a concrete property of the artifact format, not a reason to avoid reproducible packaging. Store artifacts where only trusted processes can write them, and check a hash before loading.

A prediction-equivalence check catches missing transforms and schema-order errors: keep a small reference batch (for example 20 rows covering every formation and a few missing values) with the predictions the model produced at training time, and assert that a freshly loaded artifact reproduces them.

The metadata file answers the questions someone will ask in a year: which columns in which units and order, which library versions, which training data version and date range, which decision threshold and why. The model card, next slides, adds the human context: intended use and limitations. Pickled scikit-learn models are only guaranteed to load with the same library version; pin versions in the deployment environment. Transition: the code.

Persist the pipeline and pin the versions

PIPELINES, LEAKAGE, AND DEPLOYMENT

Persist the pipeline and pin the versions

```python
import json, sys, joblib, sklearn, numpy as np

joblib.dump(model, "rop_model_v3/pipeline.joblib")
meta = {
    "features": numeric_columns + categorical_columns,
    "target": "ROP (m/h)",
    "sklearn": sklearn.__version__,
    "numpy": np.__version__,
    "python": sys.version.split()[0],
    "training_data": "rop_wells_2026-09 (wells D1-D5)",
}
with open("rop_model_v3/metadata.json", "w") as f:
    json.dump(meta, f, indent=2)

reloaded = joblib.load("rop_model_v3/pipeline.joblib")   # trusted files only
assert np.allclose(reloaded.predict(X_ref), expected_ref)
```

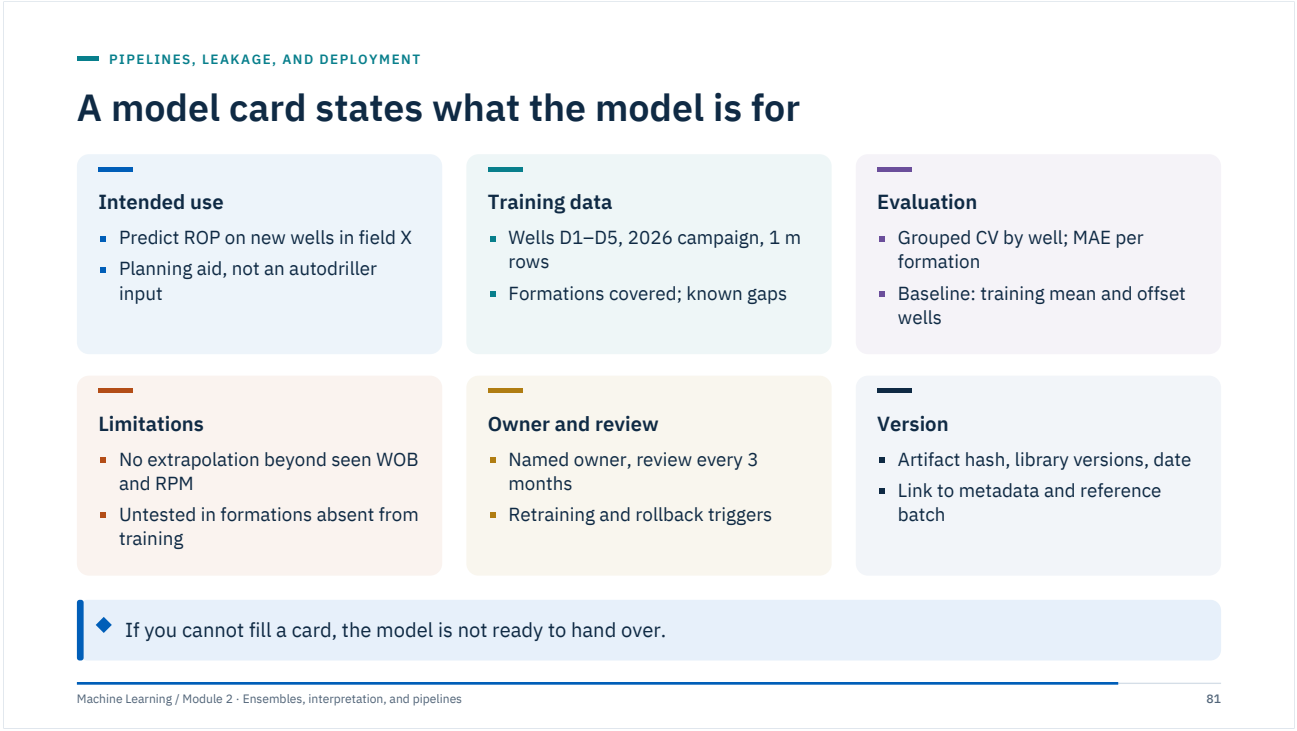
Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 80

[3 min] Context: model is the fitted search.best_estimator_ from the previous slide, the column lists are those of the ColumnTransformer, the folder rop_model_v3 exists, and X_ref with expected_ref is the stored reference batch. Walk through the three blocks. First, joblib.dump saves the fitted pipeline: imputers, encoder, and model in one object. Second, a small JSON file records the schema and the versions that produced the file; for the environment that produced this deck's reference results that is scikit-learn 1.7.2, NumPy 1.26.4, and Python 3.12. Third, the reload check: in a fresh process, load the artifact and assert that it reproduces the stored predictions for the reference batch X_ref.

Pinning: record exact versions in a requirements or lock file next to the artifact and install the same versions wherever the model runs. scikit-learn warns when unpickling a model saved with a different version; treat that warning as an error in production. For long-lived or cross-platform deployment, formats such as ONNX, or the skops library's safer persistence format, are alternatives worth evaluating.

Common error: saving only the final estimator (model.named_steps["forest"]) and re-implementing preprocessing in the serving code, which almost always drifts from training. Transition: the model card.

A model card states what the model is for



[3 min] Model cards (Mitchell and colleagues, 2019) are short documents that travel with a model. The six cards here are an adapted template for engineering models; the example content refers to our synthetic ROP model and is illustrative.

Intended use states what decisions the model supports and, just as importantly, what it must not be used for: a planning aid is different from a closed-loop control input. Training data states the population: wells, period, formations, row definition, known gaps. Evaluation states how performance was measured, on which groups, against which baseline, with the variation across wells rather than a single number. Limitations lists where the model is expected to fail: extrapolation, missing formations, new tools. Owner and review names a person and a schedule, with the conditions that trigger retraining or rollback. Version links the card to the exact artifact.

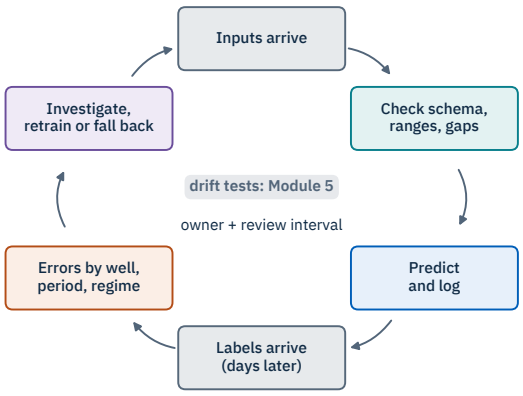
Ask each table to write the Limitations card for the facies model from Part 2 (new-well calibration offsets, thin beds). Transition: what happens after release.

Production evaluation continues after release

PIPELINES, LEAKAGE, AND DEPLOYMENT

Production evaluation continues after release

- Compare incoming sensor distributions with the training population.
- Track missingness, new categories, latency, and rejected inputs.
- When labels arrive, inspect errors by well, period, and operating regime.
- Define when to investigate, retrain, or fall back to a simpler method.



Schematic of a monitoring loop; drift tests are covered in Module 5.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 82

[3 min] Input drift alone does not prove model accuracy has changed. Conversely, performance can degrade without a dramatic marginal feature shift. Assign an owner and a review interval rather than promising automatic retraining always improves the model.

Walk the loop. Inputs arrive and are checked against the schema recorded in the metadata: types, units, ranges, missingness, unknown categories. Predictions are logged with the model version. Labels often arrive later (the facies interpretation after the well is logged and reviewed, the actual ROP after the section is drilled), and only then can errors be computed, by well, period, and operating regime, the same slices used in validation. The loop closes with a decision: investigate, retrain, or fall back to a simpler, well-understood method.

Statistical tests for drift, alerting thresholds, and retraining policies are Module 5 topics; here the point is that deployment starts a measurement process rather than ending a project. Transition: an exercise on leaks.

Spot the leaks in this workflow

PRACTICE

5 min Pairs

Spot the leaks in this workflow

```
logs = pd.read_csv("facies_logs.csv")          # 8 wells, depth-ordered rows
logs["GR_z"] = (logs.GR - logs.GR.mean()) / logs.GR.std()
logs["SAND_SHARE"] = logs.groupby("WELL").FACIES.transform(
    lambda f: (f == "sandstone").mean())
sel = SelectKBest(f_classif, k=4).fit(logs[cols], logs.FACIES)
X = logs[cols].loc[:, sel.get_support()]
X_tr, X_te, y_tr, y_te = train_test_split(X, logs.FACIES, test_size=0.2)
search = GridSearchCV(RandomForestClassifier(), grid, cv=5).fit(X_tr, y_tr)
print("test accuracy:", search.score(X_te, y_te))
```

1. Find at least four problems and say how each inflates the score.
2. Rewrite the workflow as one pipeline with a grouped split.

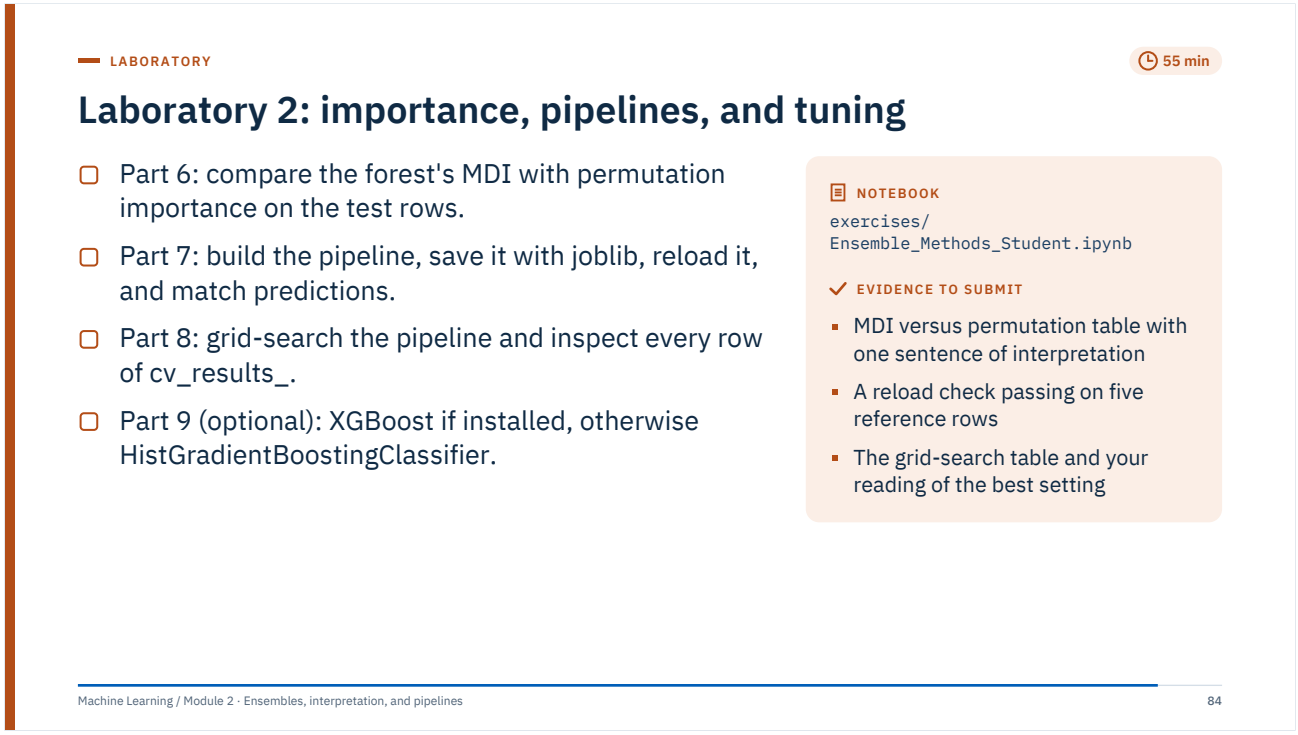
Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 83

[5 min] Assume cols lists the logs plus GR_z and SAND_SHARE. Answers, most serious first.

(1) SAND_SHARE is computed from the labels of each whole well, including test rows: target leakage. It will probably top the importance ranking, and it cannot exist for a new well. Remove it. (2) train_test_split shuffles rows, so depth-adjacent rows of the same well sit in training and test: the test score measures memorization of wells. Use GroupShuffleSplit or hold out whole wells. (3) SelectKBest is fitted on all rows, including the test rows, so the chosen features already reflect the test labels. Move it into the pipeline. (4) GridSearchCV with cv=5 splits rows, not wells, so tuning is optimistic too; use GroupKFold and pass groups. (5) GR_z uses the mean and standard deviation of all rows; for trees the effect is negligible, but the pattern is wrong: scale inside the pipeline if needed. Also: no random_state, so results are not reproducible.

Fixed shape: hold out wells for the test; Pipeline([("select", SelectKBest(...)), ("forest", RandomForestClassifier(random_state=0))]); GridSearchCV(pipe, grid, cv=GroupKFold(5)).fit(X_tr, y_tr, groups=wells_tr); score once on the held-out wells. Transition: Lab 2.

Laboratory 2: importance, pipelines, and tuning



Timing [55 min]: 5 min brief, 40 min work, 10 min debrief.

Part 6: learners plot rf_clf.feature_importances_ (MDI), then compute permutation_importance(rf_clf, X_test, y_test, n_repeats=20, random_state=0) and put both in one table. Using the test rows is acceptable here because the forest was fixed and evaluated at the end of Lab 1, but no feature may be dropped on the basis of it. Part 7: the pipeline is StandardScaler plus a forest; its CV accuracy on the training rows equals the forest's (0.950) because the scaler changes nothing for trees, so keep it only as packaging practice. Learners save it with joblib.dump, reload it in a new cell, and assert identical predictions and probabilities on X_test[:5].

Part 8: after the search, learners display pd.DataFrame(grid_search.cv_results_) sorted by rank_test_score and count the settings that share the best score; "test" in cv_results_ means the held-out fold. Section 8.1 then scores the locked choice once on the test rows. Part 9 is optional: xgboost is listed in requirements-labs-optional.txt; the cell uses the current API (eval_metric in the constructor, no use_label_encoder), falls back to HistGradientBoostingClassifier when xgboost is missing, and compares on the same folds of the training rows without reopening the test rows.

Common problems: permutation importance on training rows; calling fit on the reloaded model; reading best_params_ without looking at ties.

Lab 2 debrief: impurity and permutation disagree on Iris

DISCUSSION

Lab 2 debrief: impurity and permutation disagree on Iris

Feature	MDI (training)	Permutation (test, 20 repeats)
Petal width	0.437	0.137
Petal length	0.431	0.128
Sepal length	0.116	−0.052
Sepal width	0.015	−0.033

- Grid search: all 18 settings tie at 0.950, so best_params_ is simply the first in the grid.
- The reloaded pipeline must reproduce predictions exactly; the scaler did not matter.

[10 min] Values from the solution notebook run with scikit-learn 1.9 and the notebook's seeds.

Importance: both methods agree that the petal measurements carry the signal. MDI gives sepal length 0.116; permutation on the 30 test rows gives −0.052 (standard deviation 0.017): shuffling it slightly improved accuracy by chance. Ask why: sepal length is strongly correlated with petal length (r = 0.87), so trees split on it in training, while on held-out rows the petal features already carry that information; with 30 test rows, one or two predictions decide these values. The Part 5 lesson holds even here: MDI credits training splits; permutation measures held-out reliance, with noise that must be read.

Grid search: every one of the 18 settings has a mean CV accuracy of 0.950 (standard deviation 0.017), so the "best" parameters {max_depth: None, min_samples_split: 2, n_estimators: 50} are only the first combination in the grid. The best model scores 0.90 on the test rows, the same as the untuned forest. A tuning result is only as meaningful as the differences it found.

Close by asking which deliverable would convince a reviewer that the saved model is the evaluated model. (The reload check with a stored reference batch.)

Evidence review

PRACTICE

5 min · Groups of 3

Evidence review

The best validation score improves by 0.002 after testing 150 settings.

1. What evidence would support calling this an improvement?
2. What uncertainty comes from the search itself?
3. How would you choose if the larger model costs ten times more to serve?

[5 min] Look for paired comparisons on matched folds, independent evaluation, variability across seeds or groups, practical significance, and runtime. A tiny score increase after a large search may reflect selection noise: when 150 settings are scored on the same folds, the maximum of 150 noisy estimates is biased upward, even if all settings are equally good (the Lab 2 grid search had 18 exact ties). Do not interpret overlapping fold standard deviations as a formal significance test.

Good evidence: the same folds for both models, the per-fold differences (does the new setting win on most folds?), an untouched test or a nested cross-validation estimate (Module 5), and a check that the difference persists with other random seeds. Practical significance: is 0.002 accuracy worth anything operationally, for example fewer missed events?

Cost: with a tenfold serving cost and a negligible gain, choose the smaller model and record the decision in the model card; ask what latency and memory the wellsite hardware allows. Transition: key takeaways.

Key takeaways

KEY TAKEAWAYS

Key takeaways

- ✓ Ensembles help when members are useful and their errors differ; correlation sets the floor.
- ✓ Forests average decorrelated deep trees; boosting adds shallow trees fitted to pseudo-residuals.
- ✓ Out-of-bag scores and random validation fractions assume unrelated rows: validate by well or time.
- ✓ Explanations describe a fitted model on given data: use held-out permutation, check support, group copies.
- ✓ Ship the whole pipeline with its metadata, a reference batch, a model card, and an owner.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 87

[3 min] Read the five statements and connect each to one slide from the day: the voting curves and the variance floor; the forest diagram and the gradient-boosting stages; the out-of-bag versus grouped bars; the random-number importance and the impossible partial-dependence points; the artifact folder and the model card.

Ask each learner to pick one statement they will apply in their own work next week and write down the dataset they will apply it to. Two or three volunteers share.

What comes next: Module 3 moves to unsupervised learning (clustering); Module 5 returns to model selection with hyperparameter search strategies, nested cross-validation, calibration, rare-event evaluation, and drift monitoring, all of which build on today's ensembles and pipelines. Transition: the exit check.

Exit check

PRACTICE

7 min · Individual

Exit check

1. Explain the role of diversity in a forest, using the variance formula.
2. Explain what the next boosting learner fits under squared loss.
3. Explain why saving only the estimator may be insufficient.
4. Name one reason an out-of-bag score can be optimistic for a new well.
5. Give one reason impurity importance can rank a noise feature highly.

[7 min] Collect written answers; they are the individual evidence for the day.

Expected answers. (1) The variance of an average of M models is $\rho\sigma^2 + (1 - \rho)\sigma^2/M$; random feature subsets and bootstrap samples lower the correlation ρ between trees, which lowers the floor $\rho\sigma^2$ that more trees alone cannot remove. (2) The residuals $y - F$ of the current model (the negative gradient of squared loss), and the tree's output is added after shrinking by the learning rate. (3) Preprocessing (imputation values, category lists, column order), the schema, the version information, and the decision threshold are part of the predictor; without them a reloaded model receives differently prepared inputs. (4) A row can be out of bag while its depth neighbours from the same well are in bag, so the forest has effectively seen that well. (5) Continuous noise offers many split points that reduce training impurity by chance in deep trees; the credit does not transfer to held-out data.

Score each answer as complete, partial, or missing; revisit the most-missed item at the start of Module 3.

References and further study

FURTHER STUDY

References and further study

- Breiman (1996) Bagging predictors; Breiman (2001) Random forests. *Machine Learning* 24 and 45.
- Freund and Schapire (1997) A decision-theoretic generalization of on-line learning (AdaBoost). *J. Computer and System Sciences* 55.
- Friedman (2001) Greedy function approximation: a gradient boosting machine. *Annals of Statistics* 29(5); Friedman (2002) Stochastic gradient boosting.
- Chen and Guestrin (2016) XGBoost; Ke et al. (2017) LightGBM; Prokhorenkova et al. (2018) CatBoost.
- Wolpert (1992) Stacked generalization; Geurts, Ernst and Wehenkel (2006) Extremely randomized trees.
- Strobl et al. (2007) Bias in random forest variable importance; Goldstein et al. (2015) ICE plots; Lundberg and Lee (2017) SHAP.
- Mitchell et al. (2019) Model cards for model reporting; Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning*, chapters 7, 10, 15, 16.
- scikit-learn User Guide: ensembles, permutation importance, partial dependence, pipelines and composite estimators, model persistence.

Machine Learning / Module 2 · Ensembles, interpretation, and pipelines 89

All references are to the original papers and official documentation. Suggested reading order for learners who want more: the scikit-learn ensemble guide (practical settings), ESL chapter 15 (random forests, including the tree-correlation argument used in Part 2) and chapter 10 (boosting), then Friedman (2001) for gradient boosting.

For interpretation, the scikit-learn pages on permutation importance and partial dependence include worked examples of the pitfalls shown today (high-cardinality bias, correlated features). Lundberg and Lee (2017) introduces SHAP; Apley and Zhu (2020, Journal of the Royal Statistical Society Series B) introduces accumulated local effects as an alternative to partial dependence under correlated inputs.

For the libraries, use the official documentation of the installed version: xgboost.readthedocs.io, lightgbm.readthedocs.io, and catboost.ai/docs. APIs, defaults, and categorical support change between versions, so do not rely on blog posts for parameter meanings.

Reference: https://scikit-learn.org/stable/modules/ensemble.html ;
https://scikit-learn.org/stable/modules/permutation_importance.html ;
https://scikit-learn.org/stable/modules/partial_dependence.html ;
https://scikit-learn.org/stable/modules/compose.html ;
https://scikit-learn.org/stable/modules/model_persistence.html